

Spring 3-2022

Towards an Efficient Multi-Cloud Observability Framework of Containerized Microservices in Kubernetes Platform

Haytham Mohamed

Follow this and additional works at: <https://scholar.dsu.edu/theses>

Recommended Citation

Mohamed, Haytham, "Towards an Efficient Multi-Cloud Observability Framework of Containerized Microservices in Kubernetes Platform" (2022). *Masters Theses & Doctoral Dissertations*. 401.
<https://scholar.dsu.edu/theses/401>

This Dissertation is brought to you for free and open access by Beadle Scholar. It has been accepted for inclusion in Masters Theses & Doctoral Dissertations by an authorized administrator of Beadle Scholar. For more information, please contact repository@dsu.edu.

DAKOTA STATE UNIVERSITY

**TOWARDS AN EFFICIENT MULTI-CLOUD
OBSERVABILITY FRAMEWORK OF CONTAINERIZED
MICROSERVICES IN KUBERNETES PLATFORM**

A doctoral dissertation submitted to Dakota State University in partial fulfillment of the
requirements for the degree of

Doctor of Philosophy

in

Information Systems

March, 2022

By

Haytham Mohamed

Dissertation Committee:

Dr. Omar El-Gayar

Dr. David Bishop

Dr. Yong Wang

DISSERTATION APPROVAL FORM

We certify that we have read this dissertation and that, in our opinion, it is satisfactory in scope and quality as a dissertation for the degree of Doctor of Philosophy in Information Systems.

Student Name: Haytham M Mohamed

Dissertation Title: Towards Efficient Multi-Cloud Observability Framework of Containerized Microservices in Kubernetes Platform.

Dissertation chair: Dr. Omar El-Gayar Date: March 30, 2022

Committee member: Dr. David Bishop Date: March 30, 2022

Committee member: Dr. Yong Wang Date: March 30, 2022



Dakota State University
Completion of Dissertation Final Defense Form

Student Name: Haytham M. Mohamed

Student ID: A00506648

Title of Dissertation: TOWARDS AN EFFICIENT MULTI-CLOUD OBSERVABILITY FRAMEWORK OF CONTAINERIZED MICROSERVICES IN KUBERNETES PLATFORM

Location of Final Defense: Zoom

Date and Time of Final Defense: March 30, 2022 4:00 PM
Date **Time**

Semester of Intended Graduation: Spring 2022

The above-named student has:

- ☒ Satisfactorily passed his/her final defense without revisions to the dissertation
- ☐ Satisfactorily passed his/her final defense with revisions to the dissertation (page 2)

Revisions due: _____

date

- ☐ Had his/her final defense deferred and will be rescheduled (Please attach letter)
- ☐ Not satisfactorily completed his/her dissertation defense (Please attach letter)

<u>Omar El-Gayar</u>	Omar El-Gayar	March 30, 2022
Signature	(co-chairperson)	Date
Signature	(co-chairperson)	Date
<u>David Bishop</u>	David Bishop	March 30, 2022
Signature	(member)	Date
<u>Dr Yong Wang</u>	Dr Yong wang	March 30, 2022
Signature	(member)	Date
Signature	(member)	Date
Signature	(member)	Date

Completed form will route automatically to:
Dakota State University
Office of Graduate Studies and Research
FAX: 605-256-5093 PHONE: 605-256-5799

E-MAIL: gradoffice@dsu.edu

Name of Student:

Haytham Mohamed

Date of Final Defense:

3/30/2022

The following revisions are **required** for completion of dissertation (attach a separate sheet if necessary):

Date revisions required back to Dissertation Committee for approval: NA

Date

The following revisions are *suggested* (but not required):

Omar El-Gayar

Dissertation Committee Chairperson

Name: Omar El-Gayar

March 30, 2022

Date

Committee Co-Chairperson

Name:

Date

Haytham M. Mohamed

Student

Name: Haytham M. Mohamed

March 30, 2022

Date

Cherie Noteboom

Graduate Program Coordinator

Name: Cherie Noteboom

March 30, 2022

Date of Approval

Mark Hawkes

Dean of Graduate Programs

Name: Mark Hawkes

March 31, 2022

Date of Approval

Completed form will route automatically to:

Dakota State University

Office of Graduate Studies and Research

FAX: 605-256-5093 PHONE: 605-256-5799

E-MAIL: gradoffice@dsu.edu

ACKNOWLEDGMENT

I praise and thank Allah (SWT) for His greatness and for giving me the strength and courage to complete this dissertation and reach this milestone in my PhD degree program at Dakota State University (DSU).

I would like to thank the following people, without whom I would not have been able to complete this dissertation project, and without whom I would not have made it through my Ph.D. degree!

My dissertation committee at DSU, especially my committee chair Dr Omar El-Gayar, whose insight, intensive knowledge, and reviews steered me through this dissertation to accomplish producing a successful project. Your great support and guidance Dr. Omar throughout my PhD program at DSU allowed my studies to go the extra mile (please pardon me for all extra work and time!). And special thanks to my committee members, Dr David Bishop and Dr Yong Wang, for your great advice, and help guiding and reviewing my dissertation project.

And my biggest thanks to my family for all the support you have shown me through this dissertation and throughout the years of the program. For my kids, sorry for being super busy whilst I work on this research. For my parent, my father Mahmoud and my mother Monira, thank you for your encouragement, prayer, support, and good wishes. And for my wife Eyman, special thanks for all your support, I simply couldn't have done this without you. You have been amazing, and I will now clear all the papers off my office table and hang the curtains around the house as I promised!

ABSTRACT

A recent trend in software development adopts the paradigm of distributed microservices architecture (MA). Kubernetes, a container-based virtualization platform, has become a de facto environment in which to run MA applications. Organizations may choose to run microservices at several cloud providers to optimize cost and satisfy security concerns. This leads to increased complexity, due to the need to observe the performance characteristics of distributed MA systems.

Following a decision guidance models (DGM) approach, this research proposes a decentralized and scalable framework to monitor containerized microservices that run on same or distributed Kubernetes clusters. The framework introduces efficient techniques to gather, distribute, and analyze the observed runtime telemetry data. It offers extensible and cloud-agnostic modules that can exchange data by using a multiplexing, reactive, and non-blocking data streaming approach. An experiment to observe samples of microservices deployed across different cloud platforms was used as a method to evaluate the efficacy and usefulness of the framework. The proposed framework suggests an innovative approach to the development and operations (DevOps) practitioners to observe services across different Kubernetes platforms. It could also serve as a reference architecture for researchers to guide further design options and analysis techniques.

DECLARATION

I hereby certify that this dissertation constitutes my own product, that where the language of others is set forth, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions, or writings of another.

I declare that the dissertation describes original work that has not previously been presented for the award of any other degree of any institution.

Signed,

Haytham Mohamed

TABLE OF CONTENTS

DISSERTATION APPROVAL FORM.....	II
ACKNOWLEDGMENT	III
ABSTRACT.....	IV
DECLARATION	V
TABLE OF CONTENTS	VI
LIST OF TABLES.....	VIII
LIST OF FIGURES.....	IX
INTRODUCTION	1
BACKGROUND OF THE PROBLEM	1
STATEMENT OF THE PROBLEM.....	2
OBJECTIVES OF THE DISSERTATION	2
LITERATURE REVIEW	4
RESEARCH METHODOLOGY	7
THE DECISION GUIDANCE MODELS (DGM)	8
EVALUATION	10
THE FRAMEWORK.....	11
DATA SOURCE MODULE	11
DATA MANAGEMENT MODULES	14
DATA ANALYSIS MODULES	18
IMPLEMENTATION AND EVALUATION	21
FRAMEWORK IMPLEMENTATION.....	21
BENCHMARKING AGAINST REQUIREMENTS	21
EXPERIMENTAL EVALUATION	23
EXPERIMENTAL RESULTS	25
DISCUSSION.....	1
CONCLUSIONS.....	4
REFERENCES	6

APPENDIX A: USERS' MANUAL	9
GLOSSARY	9
HOW TO DEPLOY	9
FRAMEWORK EXPERIMENT SETUP.....	10
<i>Artifacts Deployed in Digital Ocean.....</i>	<i>11</i>
<i>Artifacts Deployed in Google Cloud Platform.....</i>	<i>12</i>
HOW TO USE THE FRAMEWORK MODULES	20
<i>DSM: Data Source Module.....</i>	<i>20</i>
Raw Spans:.....	20
Raw Logs	20
Raw Metrics	20
<i>DMM: Data Management Module.....</i>	<i>20</i>
<i>SAM: Data Stream Analysis Module.....</i>	<i>22</i>
<i>DIM: Data Inspection Module</i>	<i>24</i>
APPENDIX B: SYSTEM TECHNICAL DOCUMENTATION	25
FRAMEWORK MODULES	25
DATABASE SCHEMA	25

LIST OF TABLES

Table 1: DGM Requirements for Microservices Monitoring System	9
Table 2: Observability Framework Instantiated Artifacts	11
Table 3: Mappings of Designed Modules to Requirements.....	23
Table 4: Platform Runtime Performance	26

LIST OF FIGURES

Figure 1: Research Methodology per DSR Process Model	7
Figure 2: Design Areas in the Decision Guidance Model	8
Figure 3: DGM Elements and Relationship	9
Figure 4: Data Generation Design Options	12
Figure 5: Data Generation with Agents	12
Figure 6: Resolving Microservice Name from Kubernetes	14
Figure 7: Monitoring Data Distribution	15
Figure 8: Data Flow Between Modules	15
Figure 9: Monitoring Data Storage	16
Figure 10: Storage of Raw Data.....	16
Figure 11: Storage of Aggregated Data	16
Figure 12: Monitoring Data Hosting.....	17
Figure 13: Telemetry Database Schema	17
Figure 14: Telemetry Data Processing Design Options.....	18
Figure 15: SAM for Data Aggregation Service	19
Figure 16: Anomaly Detection and Correlation Based on Data Stream	20
Figure 17: Anomaly Detection and Correlation Based on Stored Data	20
Figure 18: Anomaly Correlation with Other Telemetry Data.....	20
Figure 19: Framework Multi-Cloud Instantiation.....	24
Figure 20: SkyWalking CPU Utilization Rate	26
Figure 21: Modules CPU Utilization Rate	26
Figure 22: SkyWalking Memory Utilization Rate	26
Figure 23: Modules Memory Utilization Rate	26
Figure 24: SkyWalking Threads by State	27
Figure 25: SAM Module Threads by State	27
Figure 26: DMM Module Threads by State.....	27
Figure 27: DSM Module Threads by State	27
Figure 28: Computed Samples in SkyWalking.....	27
Figure 29: Computed Sample by Proposed Modules	28
Figure 30: Framework Modules Tasks	2

Figure 31: Deployment Artifact.....	10
Figure 32: Experiment Setup	11
Figure 33: DOCP - 5 nodes x 4 vCPU x 16 GB RAM	11
Figure 34: GCP - 5 nodes x 4 vCPU x 16 GB RAM	11
Figure 35: DMM Units Storing RAW Data.....	14
Figure 36: Backend service raw metrics	14
Figure 37: Backend service logs	14
Figure 38: Backend service invocation traces	14
Figure 39: Middle service raw metrics	14
Figure 40: Middle service logs	14
Figure 41: Middle service invocation traces	14
Figure 42: Frontend service raw metrics	15
Figure 43: Frontend service logs.....	15
Figure 44: Frontend service invocation traces	15
Figure 45: Microservices Captured Metrics	15
Figure 46: Microservices Captured Metrics	15
Figure 47: Microservices Captured Metrics	15
Figure 48: DMM Storage of Aggregated Data	16
Figure 49: Aggregated Metrics Ingestion Rate	17
Figure 50: Sample of the Stored Aggregated Metrics	17
Figure 51: Framework Modules and Capabilities.....	19
Figure 52: Example of Stored Raw Metrics data.....	21
Figure 53: Example of Stored Event Logs data	21
Figure 54: Example of Stored Service Interaction Traces and Spans Data	21
Figure 55: Example of Stored Summarized Metric Data.....	22

CHAPTER 1

INTRODUCTION

Background of the Problem

The emerging paradigm of distributed microservice architecture is widely adopted by organizations to transform business critical software applications into modern architecture. In a microservices pattern, a monolithic application is decomposed into several distributed and interconnected services. Each service is purposely fit to serve specific domain functions that pertain to a business context. In tandem with recent advancements in cloud computing technologies and services, the adoption of this microservices design pattern has increased momentum. The introduction of containerization technology, such as Docker, has paved a flexible way to package, distribute, and run microservices in self-contained images. Each image includes the dependencies and runtime needed to execute and instantiate a container. However, many possible factors may cause a running container to fail. Issues in network communication, degrading service performance or resource constraints in a runtime environment may be a culprit. Failing containers need to re-instantiate to reliability host microservice workloads.

The introduction of orchestrating platforms such as Kubernetes, has come to the rescue. Kubernetes is the prevalent orchestration platform chosen by many organizations to host and run containerized microservices due to its maturity, robustness, and rich extensible features. The platform offers a mechanism to host distributed instances of a microservice on a cluster of several infrastructure nodes. Each instance runs as a unit of deployment, called a Pod. Kubernetes safeguards against availability risks. If a Pod dies for any reason, Kubernetes can instantiate and run a new one to keep the number of running instances at the desired number.

Maintaining a reliable production-grade microservices requires a way to observe the behavior of running instances of a microservice to avoid potential risk factors. However, despite the benefits of using a cloud platform, it becomes challenging to observe the performance characteristic of complex applications (Fazio et al., 2016). It gets more challenging in scenarios wherein microservices are distributed on multi-cloud Kubernetes environments. Organization may choose a multiple-cloud hosting scenario to shield applications from outages while gaining

scalability, satisfying utilization demand, and maintaining competitive operational costs. Furthermore, the distributed and ephemeral nature of Kubernetes environments exacerbates the role of observability and poses novel challenges to monitoring data, such as event logs, performance metrics, and interaction traces (Cinque et al., 2019b). Therefore, it is essential to have a monitoring framework to collect, distribute, and analyze monitoring data to gain insights and inform guided observability decisions. The aim of this design is to devise a distributed and decentralized observability framework to effectively observe microservices that run in multiple Kubernetes cloud platforms.

Statement of the Problem

A business-critical application that is decomposed to many distributed microservices instances hosted in Kubernetes cloud platforms is ephemeral in nature. A microservice instance is subjected to failure risks due to various run-time or environmental factors that could affect its availability to serve requests. It is important to be able to monitor and observe how well microservices are performing. The complexity involved in observing distributed systems across different cloud platforms has built a consensus support for a need for linearly scalable monitoring systems (Rovnyagin et al., 2018). The important realization of such observability needs has motivated this research to investigate “*how to efficiently observe microservices-based applications that run in distributed multi-cloud Kubernetes platform*”.

Objectives of the Dissertation

The work in this dissertation leverages recent advancements in stream communication and processing techniques to contribute the following:

- Designing a scalable and distributed cloud system to collect and transfer telemetry data of distributed microservices across different cloud platforms
- Equipping the framework with management and analysis capabilities on the observed telemetry data.
- Processing observed data to detect anomalies on a time series signal of a performance metric.

The framework proposed in this dissertation provides a scalable and efficient cloud system to software development and operation teams to observe complex distributed microservices-based applications. Software development and operation practitioners can use this innovative mechanism to gather, distribute, and compute aggregates to analyze and gain insights from the collected telemetry data. Furthermore, the presented design guides researchers with a reference architecture to extend and provide more observability and analytical capabilities

CHAPTER 2

LITERATURE REVIEW

Organizations have used containers technology to gain agility and accelerate building software that can easily be modified based on ever-changing business needs. Containers are used to run a variety of microservices-based applications, such as long running services, batch processing, internet of things, and artificial intelligence. Adoption of microservices to design complex application is increased because of its applicable pattern to manage upgrading and scaling while handling increased users demands. A complex application can be formed as a set of stateless units of independent services that communicate and collaborate with each other to serve a certain use case request. Each autonomous service unit is referenced in this context as a microservice and is usually packaged as a container (Khan, 2017).

Containers provides low overhead and speed to run and scale microservices. However, maintaining a stable deployment of complex applications at scale that run with ephemeral containers raises the need for orchestrating platforms. These platforms provide a mechanism to automatically provision, orchestrate, and scale running containers (Al Jawarneh et al., 2019). There are several platforms available to manage orchestrating containerized microservices, such as Kubernetes, Mesosphere, Docker Swarm, and Nomad (Khan, 2017). These orchestrators can manage to host complex microservices applications in cloud platforms with clusters composed of several infrastructure nodes to achieve high availability.

A complex application of containerized microservices deployed on orchestrating platform, such as Kubernetes, has diverse runtime characteristics that requires extensive monitoring. Monitoring microservices is an important task that is necessary to ensure the quality of service of applications hosted on cloud platform (Alhamazani et al., 2014). However, the dynamic, distributed, and diverse composition of microservices applications has introduced challenges to this monitoring task. These challenges have motivated several efforts to devise techniques to observe, collect, and investigate microservices runtime telemetry.

For instance, a framework was introduced to continuously extract quantitative metrics from passive traces of intercepted network packets traveling between application's microservices nodes (Cinque et al., 2019a). This framework proposed an unintrusive technique

to collect and analyze traces and logs. However, it has no emphasis on the collection and analysis of microservices runtime performance metrics. It is also not applicable to microservices hosted in Kubernetes platforms on multi-cloud environments. Another effort was made to introduce a cross-layer multi-cloud monitoring system (CLAMS) to collect and share quality of service (QoS) information in the cloud (Alhamazani et al., 2014). However, this proposed framework aimed to only monitor Virtual Machines (VMs) and containerized microservices. One more attempt was made to design a cloud computing architecture to monitor and process high-volume telemetry (Rovnyagin et al., 2018). This design depends on leveraging extra messaging middleware to distribute the observed data. A recent work suggested an agent-based framework to monitor microservices in heterogeneous cloud platforms (Noor et al., 2019). This design used agents to send data to a central global manager over HTTP that may pose as a single point of failure (SPOF). The proposed agents communicate periodically with a manager to check updates in flow rate configuration. These frequent checks subject the proposed framework to a chatty network of communication between agents and the central manager. Furthermore, the proposed framework dealt with microservices performance metrics and did not consider other telemetry data such as logs and traces.

Besides the previously mentioned contributed frameworks, many cloud providers offer observability services, such as Amazon CloudWatch (Amazon, 2022), Google Cloud Monitoring (Google, 2022), and Azure Monitor (Microsoft, 2022). However, these services target the prospective cloud platform of the provider. Also, there are frameworks and tools such as Prometheus (Prometheus, 2022), Datadog (Datadog, 2022), and OpenTelemetry (OpenTelemetry, 2022). While these tools can monitor applications in several platforms, they provide limited functionalities and rely on external systems to perform complex data analysis tasks (Noor et al., 2019). Furthermore, these tools are not suited for tasks to transfer and analyze telemetry data of microservices that run in multiple cloud environments. Furthermore, Apache SkyWalking (SkyWalking, 2022) is one of the frameworks that is designed for cloud-native and containerized microservices that run in platforms such as Kubernetes. Apache SkyWalking can monitor and perform complex telemetry data computations, but it is not possible to apply it as a decentralized cluster across multiple cloud platforms.

Shortcomings in prior proposed observability frameworks has inspired the work in this dissertation to address the outlined gaps. The aim is to design a distributed, decentralized, and

extensible system to monitor microservices that are deployed in multiple Kubernetes cloud platforms. Specifically, the designed framework enables the acquisition of monitoring data, and the provision of efficient computation methods while avoiding single point of failure (SPOF) risks. Furthermore, the proposed framework can publish and analyze data of all types (performance metrics, event logs and interaction traces) without a need to introduce message queues middleware components.

CHAPTER 3

RESEARCH METHODOLOGY

This dissertation uses the design science research (DSR) methodology while guided by the Decision Guidance Models (DGM) to systematically explore, design, implement, evaluate, and document the proposed framework. As design science is concerned with “devising artifacts” to attain goals (March & Smith, 1995), the contributed design offers artifacts in the forms of methods and instantiation to monitor microservices across different cloud Kubernetes platforms. The framework introduces innovative methods to transfer and analyze the monitoring telemetry data and instantiates modules that implement the introduced technique.

Design science innovatively addresses important unresolved problems; or offers a more effective or efficient ways to tackle existing problems (Hevener et al., 2004). The complexity and challenges in monitoring microservices across different cloud Kubernetes platforms has motivated and initiated activities to search for innovative solution. Based on the existing design science research process model (Vaishnavi & Kuechler, 2004), and as illustrated in Figure 1, a literature review was conducted to identify important requirements and design options in this problem domain. Accordingly, a tentative design was introduced with identified essential modules. Suitable techniques that realize the framework functions were used to implement the modules. Experimental and descriptive methods were used to evaluate the efficacy (utility) of the contributed framework. Further, the proposed utilities in the framework were compared with equivalent functions that are featured in an existing framework to assess efficiency. Finally, the design process concluded with a summary of knowledge learned, design contributions, and suggestions for further investigation.

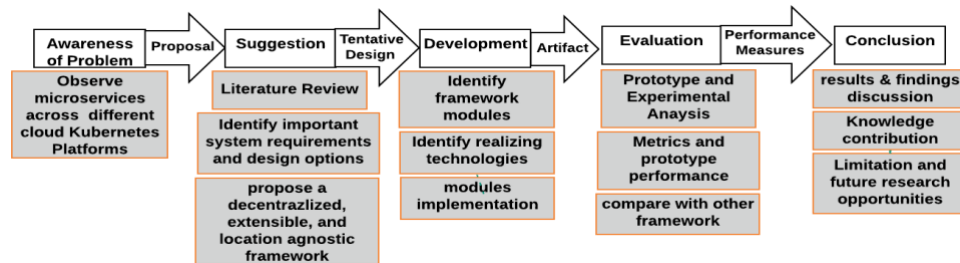


Figure 1: Research Methodology per DSR Process Model

The Decision Guidance Models (DGM)

Decision Guidance Models (DGM) is a structured way of describing software architectures (Rovnyagin et al., 2018) and is used for design space exploration and documentation (Haselböck et al., 2017). Many research works used DGM method, such as decision guidance for microservice monitoring (Haselbock & Weinreich, 2017), decision guidance for microservices discovery and fault tolerance (Haselböck et al., 2017), and decision model for cyber-foraging systems (Lewis et al., 2016).

The DGM method starts with identifying important requirements and design options to devise modules that compose an observability framework. Existing literature in this space has defined four essential activities to operate a monitoring system. These activities are *data collection, processing, dissemination* and *presentation* (Mansouri-Samani & Sloman, 1993). Further work introduced DGM with four corresponding major areas to monitor microservice-based systems (Haselbock & Weinreich, 2017). As illustrated in Figure 2, these four areas are *data generation, management, processing, and dissemination*.

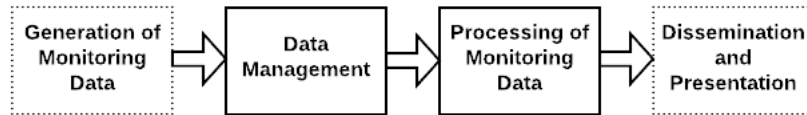


Figure 2: Design Areas in the Decision Guidance Model

Each DGM area encompasses a set of requirements that are typically served by a microservices monitoring system. As summarized in Table 1, the data generation area (R-1) aims to produce telemetry, such as resource consumption (CPU / Memory), from platform and services levels, in addition to errors rate, throughput, event logs, and interaction traces from services level. Usually, the generation of monitoring data is achieved by using agents or embedding libraries to instrument hosts, runtime environment or services. The data management area (R-2) concerns with using either a pull or a push method to distribute monitoring data, determine whether to utilize centralized or decentralized data storage, and determine the location to host the data either in the cloud or on-premises. The collected telemetry needs to be analyzed (validated / filtering / summarized / aggregated) in the data processing phase (R-3), and alerts to be raised if a telemetry violates a defined threshold. Finally, the design area of data dissemination and presentation (R-4) targets the need to present

the collected and summarized data in consumable formats that is easy to understand by stakeholders (such as dashboard, graphs, and tables).

Many agents and libraries are readily available for generating monitoring data (R-1) from platforms and services. Applications with dashboards to present reports and plot graphs of monitored data (R-4) are also available to use. Accordingly, this research focuses on the design of modules that realize requirements that pertain only to areas of data management (R-2) and the processing of monitoring data (R-3).

Table 1: DGM Requirements for Microservices Monitoring System

Requirement	DGM Area of Design	Functional Requirements
R-1	Generation of Monitoring Data	Instrumenting hosts, runtime environments, and services by placing probes and sensors to record events relates to: - Information on the runtime infrastructure (CPU/ Memory/consumption). - Services metrics (response time / failure rate / throughput/ CPU / Memory). - Services interactions (communication path traces).
R-2	Data Management	Distributing collected telemetry data (pull or push), determining storage location (centralized or decentralized), and hosting strategy (self-hosted or SaaS).
R-3	Processing of Monitoring Data	Combining data, filtering, and performing data aggregation and alerting.
R-4	Dissemination and Presentation of Monitoring Data	Reporting, dashboarding, and dissemination to stakeholders.

Furthermore, referenced DGM annotations (Haselbock & Weinreich, 2017) are used to graphically describe different aspects of the framework design. As depicted in Figure 3, the colored annotations illustrate area of concerns, chosen design options with implementing technology to address the concerns, and any potential implications caused by the chosen design options.

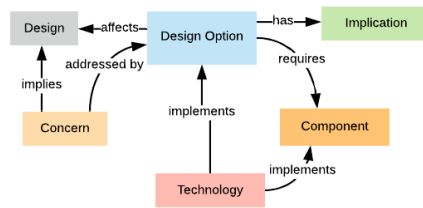


Figure 3: DGM Elements and Relationship

Evaluation

In the evaluation phase, the framework was prototyped to verify the efficacy and efficiency of the designed modules. The efficacy experiment evaluates the framework intended observability utilities such as collecting, distributing, computing summarized metrics, and detecting threshold-based anomalies with correlated telemetry. The designed modules of the framework and a workload of illustrating microservice samples were deployed across two of Kubernetes clusters, each was hosted in a separate cloud platform. The illustrating workload of microservices were configured to publish runtime and system metrics to be scraped, write event logs to standard output, and send invocation traces to collecting agents. The collecting agents feed the framework with telemetry data on demand. Furthermore, the framework efficiency was evaluated by comparing its performance characteristics in terms of resources consumption such as memory, CPU, and number of threads used against an existing framework of similar functionalities. Apache SkyWalking was chosen as the benchmarking observability framework due to its applicability to observe microservices and perform complex computation tasks in Kubernetes environments.

CHAPTER 4

THE FRAMEWORK

The framework is composed of supporting components each addressing a certain area of the requirements in the model. In particular, the design includes five modules that are summarized in Table 2 as a *data source module* (DSM) to handle receiving generated data, a *data distributed module* (DSM) to distribute data, a *data management module* (DMM) to handle storing and retrieving data, a *data streaming analysis module* (SAM) to process and compute data stream, and a *data inspection module* (DIM) to inspected anomalies and correlate data. The following sections explain the design of each of these components.

Table 2: Observability Framework Instantiated Artifacts

Modeling Construct	Description
Data Source Module	DSM - Framework module to receive data from collecting agents.
Data Management Modules	DDM and DMM - Framework modules to manage distributing and persisting data.
Data Analysis Modules	SAM and DIM - Framework modules to perform data streaming computation and inspection.

Data Source Module

Data generated from continuous detection of important logged events, runtime characteristics and traces in monitored systems represent historical views of system activities (Mansouri & Sloman, 1987). In microservices systems, these views are based on time series of generated telemetry data as either performance metrics, event logs, or activity traces. Services implementation may log out messages of certain important events. Containerized microservices commonly write out event logs to standard output. Also, options are possible to either manually

or dynamically instrument services code to capture traces of interactions that flow between services. Finally, runtime performance characteristic could be scraped from microservices runtime and underneath the runtime platform as an option to obtain metrics data.

Several techniques are available to generate telemetry data from the runtime environment or by instrumenting services. Microservices typically use available packages to write out logging messages. Libraries are also available that could be embedded or run adjacent to a service to intercept calls and record interaction traces. Furthermore, services and platform usually cater their performance characteristic information through managing components or an interface. As illustrated in Figure 4, mechanisms are needed to acquire these different types of data and make them available for any data analysis tasks. Contemporary techniques to acquire telemetry data are offered by tools such as Prometheus, OpenTelemetry (OpenTelemetry, 2022), and Fluent-Bit (FuentBit, 2022). The framework uses these tools to feed telemetry data to the framework.

As shown in Figure 5, the framework uses OpenTelemetry (OTel) agents to route telemetry data to the framework. Agents are placed on every node in Kubernetes clusters across different cloud providers. These agents are deployed as “Daemon sets” objects in Kubernetes terms. OTel agents are configured with Prometheus to periodically scrape metrics data from distributed instances of all deployed microservices. Traces that resulted from services interactions are sent directly from the instances to the OTel agents. Log messages are captured by Kubernetes on a nodes files system. Fluent-Bit agents continuously capture and forward these messages to OpenTelemetry agents. Hence, OTel agents become the main data source that sink collects data to the Data Source Module (DSM) of the framework.

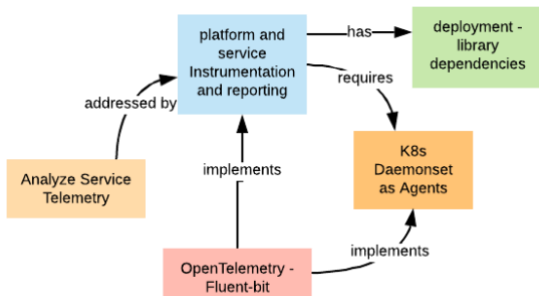


Figure 4: Data Generation Design Options

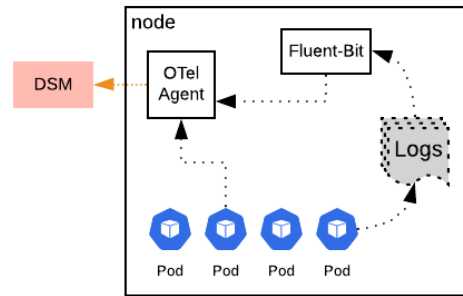


Figure 5: Data Generation with Agents

The DSM can receive data from OTel agents by using HTTP or a gRPC message protocol. Consuming modules can request all or a subset of telemetry data stream from the

DSM. A request could be used with a query expression to filter the telemetry data stream. The framework offers the ability to filter data by metric name, service name, and an optional set of qualifying telemetry labels. For instance, sending a request to the DSM with a query that looks like “*jvm_memory_used_bytes@backend-service*” would obtain a metrics stream of the memory used, in bytes, by a service named “*backend-service*”. Logs and spans data streams could be filtered by using the same query expression format that starts with “logs” and “spans” respectively.

The potential implications from this approach of receiving monitoring data include the need to deploy agents in Kubernetes cluster. These agents consume runtime resources, such as CPU and memory that need to be accounted for in Kubernetes cluster capacity. Furthermore, the solution chooses to use libraries with the ability to link logs messages with interacting traces to help with the analysis. Some framework libraries may not have such features and may depend on external tools to perform such linkage. Also, consideration should be given to the format of log messages and to configure FluentBit to properly parse texts into a structured form.

The DSM converts all collected telemetry types into a unified data model named as “*TelemetryDataPoint*”. The model keeps common telemetry data elements such as timestamp, value, and labels. The DSM also extracts useful data elements, such as service and service instance information from telemetry labels and makes them available in the model. The model keeps telemetry data as text-formatted values in the case of the logs data type and as numerical-formatted values in the cases of metrics and traces. Usually, telemetry data obtained from containerized service instances in Kubernetes environment do not include their routing Kubernetes service names. As illustrated in Figure 6, DSM implements a customized Kubernetes controller to map microservice Pods to the names of their serving endpoints. The DSM caches the mappings for better performance and to eliminate chattiness with Kubernetes system.

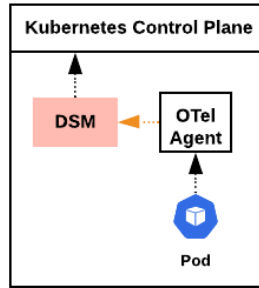


Figure 6: Resolving Microservice Name from Kubernetes

Data Management Modules

Design options to manage monitoring data were captured in three areas (Haselbock & Weinreich, 2017). One area considers data distribution options with selecting either a push or pull strategy. The second area concerns with considering either a centralized storage location or a decentralized storage location. The third area pertains to choosing either a self or managed hosting strategy to persist telemetry data.

As far as data distribution, the Framework is challenged with capturing monitoring data from several Kubernetes cloud platforms. Hence, it needs to provide an efficient mechanism to distribute the data with lowest perceived network latency as possible. The design uses the capability of reactive streaming and the backpressure mechanism to control flowing data between components. The asynchronous, reactive and binary protocol implementation of RSocket (RSocket, 2022) technique is used to efficiently distribute data across different clouds. The protocol enables flows of byte streams over a TCP transport layer and can establish reliable communications between components in the framework.

Additionally, the design uses clusters of connected RSocket broker (Gibbs et al., 2022b) instances to connect different framework components. The brokers cluster acts as a data distributor module (DDM) to route monitoring data, in a location-agnostic way, to all consuming modules. This technique enables the framework to be distributed and extensible while decentralizing the way monitoring data is shipped and consumed. RSocket brokers can enable instances of distributed data source modules (DSM) to communicate and provide data to other consuming modules across different cloud platforms. For example, as depicted in Figure 7 **Error! Reference source not found.**, a consuming module hosted in any cloud provider can request to stream monitoring data from all DSM instances across all cloud

platforms. Modules communicate with any peer by using module names and qualifying labels that are registered in the DSM cluster. Hence, communicating over DDM cluster does not require knowing the IP addresses or locations of connected modules. The RSocket-based DDM represents a mechanism by which the framework implements the design option to address DGM concern in the area of data distribution, as depicted in Figure 8**Error! Reference source not found..**

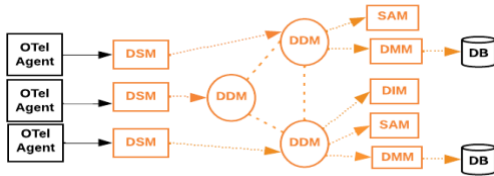


Figure 7: Monitoring Data Distribution

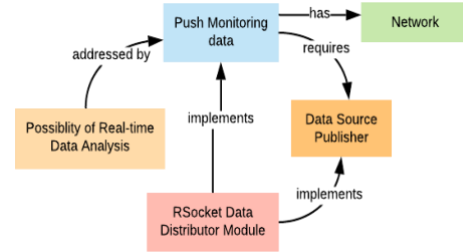


Figure 8: Data Flow Between Modules

Furthermore, the framework offers a data management module (DMM) to address the DGM concern in the area of data storage of telemetry data as annotated in Figure 9. DMM subscribes to a DDM cluster to request telemetry data from all connected DSMs. DDM can be configured to store data in raw or summarized forms. On one hand, if DDM is chosen to store raw telemetry, it would request data from connected DSMs and send that directly to storage as depicted in Figure 10. However, this option comes with the implication of requiring enough storage capacity to store such a granular and detailed level of telemetry data. The framework alleviates this drawback by storing data in tables partitioned based on data generation time. The tables were configured with retention policies to automatically remove aged data. On the other hand, to store data in a summarized form, the DDM uses configurable custom rules to compute aggregated values before sending to data storage. The rules use query expressions to filter telemetry data and use functions to evaluate aggregated values.

The framework facilitates a scalable mechanism for the DDM to efficiently compute summarized values. As depicted in Figure 11, the DDM equally distributes the configured custom rules to all connected Streaming Analysis Modules (SAM). Each connected SAM uses filters defined in its assigned rules to pull qualified data streams from all connected DSM and, in parallel, computes and sends stream of aggregated values back to the DMM. The DMM can manage to store aggregated data captured in every period duration and over a sliding time window based on computed rules. For example, a computed rule that looks like

“`http_server_requests_seconds_sum@backend-service.increase(10m).slide(5s)`” would create a request to compute the increase of response time in a service named “backend-service” during the last ten minutes and for every five seconds. The DMM can also store data with certain rate of batch size. The stored data could later be retrieved and used to perform different analysis tasks such as anomaly detection, reporting, or machine learning.

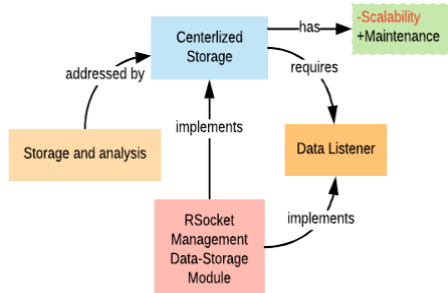


Figure 9: Monitoring Data Storage

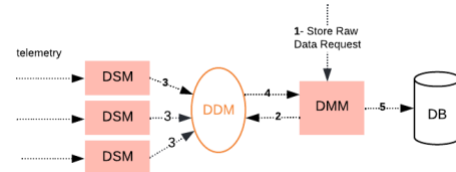


Figure 10: Storage of Raw Data

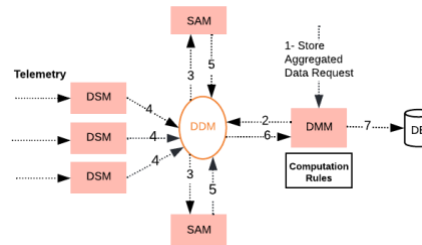


Figure 11: Storage of Aggregated Data

The DMM facilitates data retrieval through querying mechanism. A query string could be used to specify type of telemetry data, the observed service name, arbitrary qualifying labels, and aggregation function if desired. Retrieval queries result in a continuous stream of data that may include raw or summarized values (if aggregation function is specified in the query). To enable streaming of summarized values, the DMM implemented a querying mechanism, using reactive streams technique, that continuously evaluates the aggregated values of the telemetry time series data (only for metrics or span telemetry type). The pseudocode listed below illustrates, from a high level, the algorithm used by the DMM to continuously stream aggregated telemetry from the raw time series stored data.

```

INIT begin = current time
INIT status = new
FUNCTION queryData
  PASS IN: param as timestamp
  SET start = param
  SET end = current timestamp
  SET diff = end - start
  IF diff < aggregation window THEN

```

```

    Start = end – aggregation window
  END IF
  aggregate telemetry data between “start” and “end” timestamp
  IF aggregation process is started THEN
    SET status = in progress
  ELSE IF aggregation is in progress THEN
    SET begin = end
  ELSE IF aggregation is completed THEN
    SET status = new
  END IF
  PASS OUT: stream of aggregated data
END FUNCTION
INVOKE the queryData function to obtain up-to-now stream of aggregated telemetry data
WHILE data is streaming THEN
  IF stat is “new” THEN
    INVOKE the queryData function to obtain a continuous stream of aggregated telemetry data
  END IF
END WHILE
COMBINE the up-to-now data stream with the continuous data stream

```

As for the data hosting concern in the guiding DGM, as depicted by Figure 12, the framework chooses the option to store data in a cloud-hosted time-series database management system. Relational database “PostgreSQL” was used and extended with the open source “TimeScale” plugin to enable storing timeseries data. A database schema was modeled with three tables each to store one telemetry data type. Some fields are common in all three tables, such as a timestamp, value, and binary encoded data of key/value metadata labels. The value field is of a numeric type for both metrics and trace spans telemetry types, and of text type in the case of logs telemetry type. All tables were defined to have a field to store a microservice name. The trace spans and logs tables were defined to have fields for activity trace and span identifiers, and the metrics table was defined to have a metric name field. Figure 13 lists the database designed schema that shows the three telemetry tables and fields.

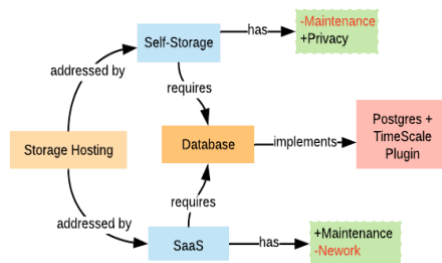


Figure 12: Monitoring Data Hosting

telemetry_metrics	telemetry_logs	telemetry_spans
ABC service	ABC service	ABC service
ABC name	ABC trace	ABC trace
JSON labels	ABC span	ABC span
ABC instance	JSON labels	JSON labels
ABC unit	ABC class	ABC parent
ABC stats	ABC thread	123 instance
⌚ time	ABC level	ABC class
123 value	⌚ time	ABC method
	ABC value	ABC url
		123 http_status_code
		ABC request_path
		ABC kind
		ABC peer
		⌚ time
		123 value

Figure 13: Telemetry Database Schema

Data Analysis Modules

The guidance model for processing monitoring data has classified design options into three general groups. These groups are service interaction analysis, aggregation of monitoring data, and threshold alerts (Haselbock & Weinreich, 2017). Service interaction analysis helps to perform root cause analysis to spot sluggish service when fulfilling requests. It entails capturing sequences of services calls along request paths to trace interactions between collaborating services. Commonly, services and runtime metrics analysis needs to acquire and retrieve monitoring data in summarized forms. Aggregated data can provide a view over time on how a service is performing, based on preconceived goals and thresholds. Deviations from these thresholds need to be observed and raised as alerts. Therefore, anomaly detection and alerting are part of the concerns that a monitoring system should address.

The design options to address concerns in the analysis area can be guided by the DGM as annotated in Figure 14. The design relies on the ability to access collected data from storage or directly from streams, either in raw or in aggregated form. The above mentioned DMM facilitates the ability to retrieve data from storage, and the DSM can supply stream of raw telemetry signals to DDM cluster subscribers. The framework offers two modules to analyze telemetry data: a data streaming analysis module (SAM) to aggregate stream of numerical telemetry values and a data inspection module (DIM) to detect anomalies and correlate telemetry to anomalies.

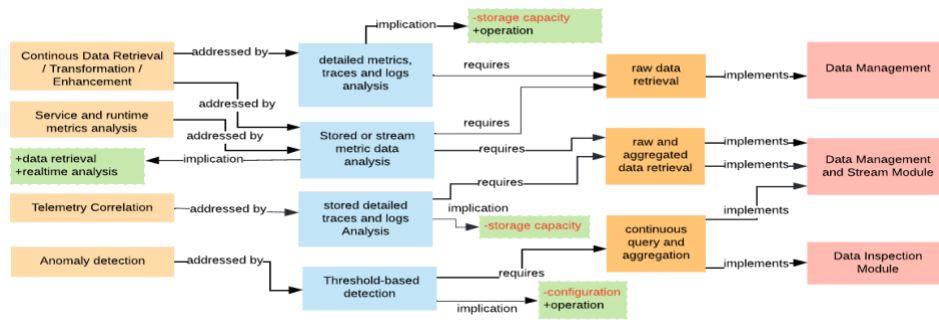


Figure 14: Telemetry Data Processing Design Options

As outlined in the previous section, the data streaming analysis module (SAM) can summarize telemetry data of numerical values. Raw Data could be continuously aggregated in real time from in-flight streams of data (hot streams). The data streaming aggregation capability offered by the SAM could be used by both the data management module (DMM) to store

summarized data and by the data inspection Module (DIM) to detect anomalies. Both the DMM and the DIM, as depicted by Figure 15, can send computation rules to SAM to continuously evaluate aggregated values.

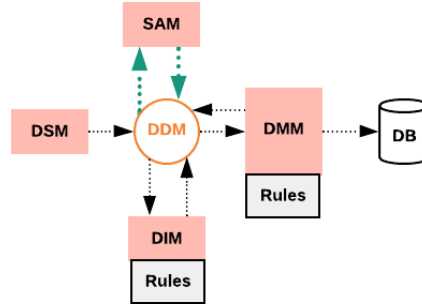


Figure 15: SAM for Data Aggregation Service

The framework implements a mechanism to configure the data inspection module (DIM) with conditional rules and thresholds to monitor and detect anomalies. The DIM alerts telemetries that violate the defined conditions. For instance a defined condition that looks like *“http_server_request_seconds_sum@backend-service.since(5m).avg(15s) > 0.002 within 5m occurrence 2 sleep 5s”* would continuously poll the number of seconds spent by requests to a service named “backend-service” in the last five minutes every fifteen seconds and raise alerts when the time spent exceeds a value of 0.002 seconds. The condition specifies to alert if the violation occurs two times within a period of five minutes. The example rule also instructs to pause for five seconds between detection evaluations to allow time for self-healing chances. The DIM can continuously detect anomalies from storage (cold streams of raw data) or in real time from in-flight streams of data (hot streams). Figure 16Figure 17 depict the sequence involve in detecting and correlating telemetry data. If a telemetry value to detect is based on real-time data streams, the DIM would route data retrieval requests to the DSM through the DDM cluster to obtain in-flight telemetry signals. And in case the detection is based on stored data; the DIM would route data retrieval requests to the DMM through the DDM cluster to obtain raw telemetry from storage. Both the DSM and the DMM facilitate ad hoc data retrieval capability to flexibly filter telemetry data regardless of its type.

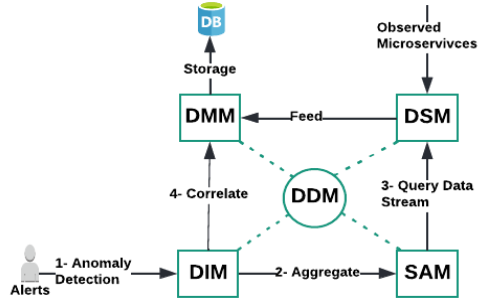


Figure 16: Anomaly Detection and Correlation Based on Data Stream

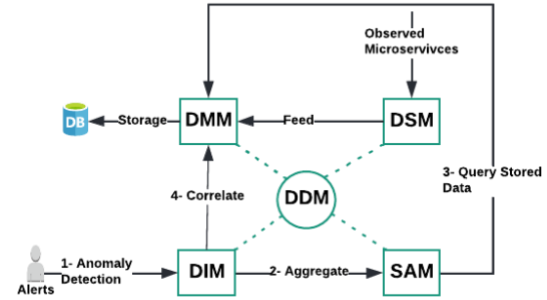


Figure 17: Anomaly Detection and Correlation Based on Stored Data

Furthermore, and as illustrated in Figure 18, when the DIM detects an anomaly in telemetry metrics data, it further tries to provide more insightful information about other related telemetry data captured at about the same time for the affected microservice. For instance, if a detected anomaly is due to a threshold violation on a metric value at time (t_i), the DIM would try to capture the stored traces and the correlated logs messages that took place in the period between $(t_i - \Delta t_i)$ and $(t_i + \Delta t_i)$. The framework provides a mechanism to configure the DIM with a value for the time offset, Δt_i .

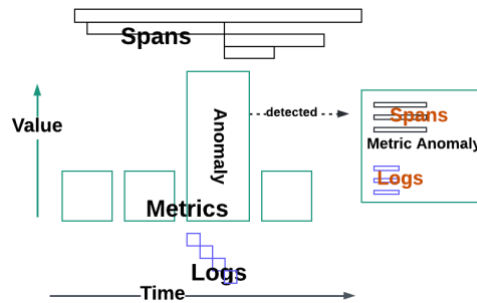


Figure 18: Anomaly Correlation with Other Telemetry Data

CHAPTER 5

IMPLEMENTATION AND EVALUATION

Framework Implementation

The proposed framework architecture was instantiated by using the Java programming language and the Spring Boot (SB) development framework. The opinionated SB framework was chosen because of its offered reactive streaming and monitoring libraries, which are needed to implement the proposed modules and illustrating microservices. It also offers mechanisms to auto-configure these libraries by setting appropriate properties. Other programming languages and development frameworks with similar capabilities could also be used to implement the framework modules.

The open source RSocket routing broker library (Gibbs et al., 2022b) is used to implement the framework data distribution module (DDM) to form a cluster of connected nodes across separate cloud platforms. The open source RSocket broker client library (Gibbs et al., 2022a) is used to implement the other modules in the framework to facilitate their communication through the DDM cluster. Furthermore, Spring libraries for Project Reactor (Reactor, 2022) helped to implement methods to stream data between modules, in addition to implementation of data streaming features such as filtering and metrics aggregation within time-windows. To implement the illustrating workload of microservices to monitor, the implementation used offered SB libraries to expose performance metrics, event logs, and trances of the services invocations. Each framework module and microservice sample was packaged as a Docker container image to deploy with manifest files in a Kubernetes platform.

Benchmarking Against Requirements

A monitoring framework must be able first to obtain the telemetry data of the observed targets. The data could either be pushed to the framework by external component or the framework itself to pull the data from those targets. The design and implementation of the data source module (DSM) facilitate a mechanism for telemetry data to be pushed to the framework. Data flow in the proposed system starts with monitored microservices that either push or pull

telemetry data to or by OTEL agents. OTEL Agents, in turn, pushes the data to DSM to enter the proposed framework. Thus, DSM is the entry point and main source for all components in the framework to obtain telemetry data.

With a mechanism to make data ready for the framework (through the DSM), the design and implementation was then able to fulfill the requirements to manage the collected data (R-2). As outlined, the DGM suggestion in the data management area is to address concerns that related to data distribution, store data in either centralized or decentralized storage, and choose between on-premises or cloud storage. The design and implementation of a cluster composed of connected instances of data distribution module facilitate a medium to distribute and transfer data to any interested module across cloud platforms. To manage data storage requirement (in R-2), the system offers the design and implementation of the data management module (DMM). The DMM can store data in either an on-premises or a cloud database, and to either a centralized or decentralized database. To illustrate the framework, the design implemented a centralized and cloud-hosted PostgreSQL database.

On the other hand, the design and implementation of the data streaming analysis module (SAM) and data inspection modules (DIM) realize the DGM requirements to process and analyze the collected telemetry data (R-4). As also outlined above, the SAM provides a way to reactively aggregate data, and the DIM can inspect telemetry data and detect violating values per configured rules. Table 3 maps the designed modules with their perspective requirement. Note that the implementation offers querying capabilities to retrieve telemetry data in raw or summarized forms. Retrieval of monitoring data helps to view and validate data flows through all the modules. Capabilities to collect telemetry data, perform analysis tasks, and query monitoring data are commonly available in typical and comparable observability frameworks, such as Apache SkyWalking. However, these frameworks fall short when compared to the proposed framework. The proposed framework offers a location-agnostic and decentralized observability modules across the cloud. For example, an instance of any module in the framework can join to consume monitoring data from any platform linked by DDM cluster. Furthermore, a module can keep up its processing speed with the pace of the data flow by applying back-pressure on the streaming data. Finally, the framework modules can collaborate to perform computation tasks by sending data streams to each other across any linked cloud platform.

Table 3: Mappings of Designed Modules to Requirements

Requirement	Implemented Artifact
R-2	Data Streaming Module (DSM)
	Data Distribution Module (DDM)
	Data Management Module (DMM)
R-3	Data Streaming Analysis Module (SAM)
	Data Inspection Module (DIM)

Experimental Evaluation

Two experiments were conducted extensively to evaluate the efficacy (functionalities) and efficiency (quality) of the designed framework. Evaluating the efficacy of the framework suggests the capacity of the architected modules to perform the intended observability tasks and produce the desired results across different cloud platforms that satisfy the required essential attributes per the guiding DGM. The details of the experiment used to evaluate the framework efficacy is described in detail in the appendix section.

Furthermore, to evaluate the efficiency of the designed framework, a second experiment was created and set up as illustrated in Figure 19. The framework modules and a workload of three simulating microservices were deployed across separate Kubernetes clusters in two different cloud environments, Digital Ocean Cloud Platform (DOCP) and Google Cloud Platforms (GCP). A Kubernetes cluster of four nodes was created in each platform. Each node has four GB in memory size and two vCPU. The workload consists of a front end, middle, and back-end services. A front-end service calls a middle service, and both were deployed in DOCP. The middle service in DOCP invokes a back-end service that ran in GCP. Framework modules, in addition to a PostgreSQL database, were deployed in DOCP. Only the DDM and the DSM instances were deployed in GCP. A DDM cluster was formed to connect the framework across the two cloud platforms. OTel and Fluent-Bit agents were deployed in Kubernetes nodes to forward telemetry data from the distributed services and runtime platforms to the framework.

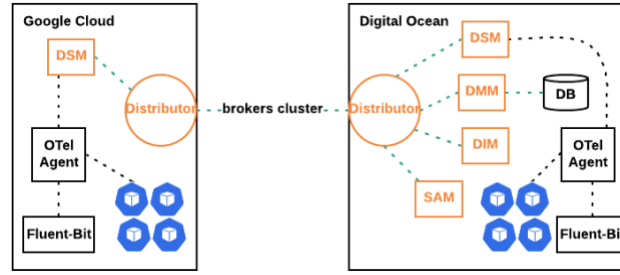


Figure 19: Framework Multi-Cloud Instantiation

The data management module (DMM) was configured with four custom metric rules to store their summarized values in a database. DMM distributes its four metric rules equally to all connected stream analysis modules (SAM) through the data distributed module (DDM) cluster to concurrently aggregate data. The SAM, in turn, multicasts requests to all connected data source modules (DSM) through the DDM cluster to acquire services and platform telemetry. After the SAM computes the aggregated values, it sends the results back to be stored in the database by the DMM.

For instance, one of the DMM computation metric rules was set as `"http_server_requests_seconds_sum@backend-service.increase(1m).slide(5s)"` to compute the latency increase in HTTP requests to a back end microservice in the last minute every five seconds. Regardless of how frequently metrics are scraped from the back end microservice, all collected values would be summarized to one value every five seconds. Because, in this case, data was summarized every five seconds, the SAM generated 12 aggregated values per minute ($60 / 5 = 12$). As there were four custom metrics to aggregate, the DMM persisted 48 records per minute ($12 * 4 = 48$). On the other side, SkyWalking framework ingested only the 4 aggregated metrics every minute.

As the designed framework uses reactive streams to perform data transfer and computations, its efficiency is compared against the existing Apache SkyWalking framework. Apache SkyWalking offers an architecture to manage collecting and analyzing telemetry and is suited to evaluate the performance of the proposed framework in a single platform. To run the comparison, both frameworks were set with the same telemetry rules to continuously compute a per-minute increase in the number of HTTP requests and duration. Also, both frameworks were instructed to save summarized telemetry to a database.

Experimental Results

The efficacy experiment with the distributed modules of the designed framework and the simulating microservices across the two-cloud environment depicted a connected cluster set up and was able to successfully transfer, filter, and process the observed telemetry data. The cloud agnostic topology enabled observing microservices regardless of the platform locations. The distributed DSM units were able to capture, transform and, on demand, transfer all three types of the observed telemetry (metrics, logs, spans) to other distributed modules of the framework. Data was successfully stored and retrieved in both raw and aggregated forms by the DMM units. And life streams of telemetry data were also successfully filtered to retrieve and aggregated. Anomalies in performance metrics based on configured threshold were detected and associated with correlated logs and spans data. This experiment supports that the designed framework can accomplish well the intended observability tasks as outlined by the guiding DGM.

Furthermore, Table 4 summarizes the performance metrics that were observed in the efficiency evaluation experiment from both the designed and the Apache SkyWalking frameworks. The performance metrics collected were CPU and memory utilization rate every five minutes, in addition to the maximum number of threads used. Note that Apache SkyWalking consumed a maximum rate of 0.03 seconds of CPU time (Figure 20) and 1 GB of memory to obtain, compute, and store the monitoring data (Figure 22). It also used a maximum rate of 10 running JVM threads and reached a maximum rate of 25 idle threads (Figure 24)

On the other hand, both the DMM and the SAM modules used less CPU rate (0.005 and 0.008 second respectively). However, the results showed that the DSM module used more CPU rate (0.045 seconds) than the Apache SkyWalking framework (Figure 21). Additionally, all proposed modules in the scenario used less memory rate. The DMM used a rate of 840 MB, and both the DMM and the SAM used 200 MB (Figure 23) Also, the number of running threads rate reached a maximum of 12 threads for both the SAM and the DMM (Figure 25Figure 26 respectively) and reached a maximum of 13 threads for the DSM (Figure 27) Furthermore, the number of waiting threads rate reached a maximum of only nine threads for the DSM module and four threads for both the DMM and the SAM modules.

It appears that the proposed DSM is the busiest component acting as a source of data to the framework. It consumed fewer system resources but was close in numbers its Apache

SkyWalking counterpart. The maximum memory utilization was only 160 BM less and the utilized running threads were only three more compared to Apache SkyWalking.

Table 4: Platform Runtime Performance

	5m CPU Utilization Rate (max)	5m Memory Utilization Rate (max)	JVM Threads (max)
Apache SkyWalking	0.03 seconds	1 GB	Running: 10, time-waiting: 25
Framework DSM	0.045 seconds	840 MB	Running: 13, time-waiting: 9
Framework DMM	0.005 seconds	200 MB	Running: 12, time-waiting: 4
Framework SAM	0.008 seconds	200 MB	Running: 12, time-waiting: 4



Figure 20: SkyWalking CPU Utilization Rate

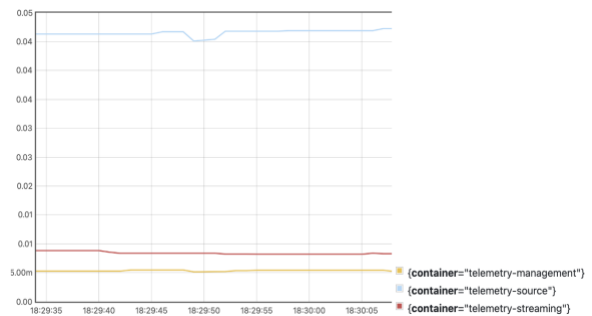


Figure 21: Modules CPU Utilization Rate



Figure 22: SkyWalking Memory Utilization Rate

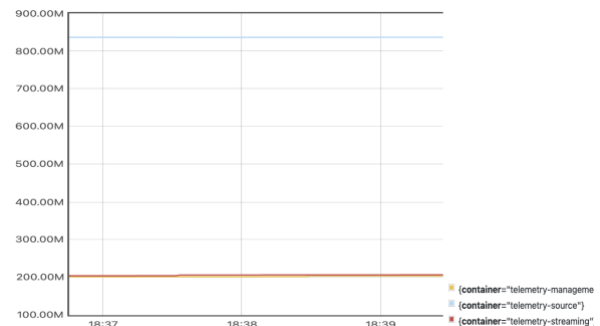


Figure 23: Modules Memory Utilization Rate



Figure 24: SkyWalking Threads by State

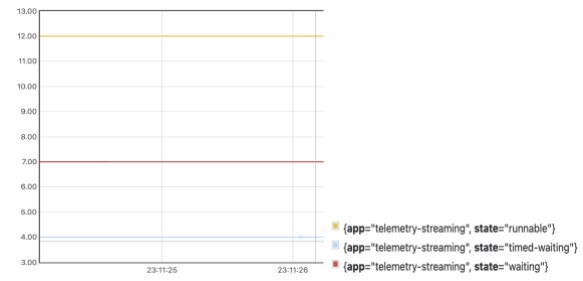


Figure 25: SAM Module Threads by State

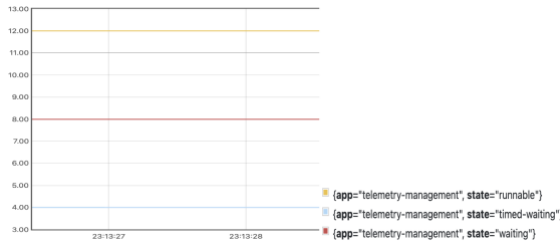


Figure 26: DMM Module Threads by State



Figure 27: DSM Module Threads by State

The retrieved sample in Figure 28 shows that Apache SkyWalking used the defined rules to compute summaries of monitoring data and saved that every minute. While the sample in Figure 29 indicates that the defined rules were evaluated by the proposed framework to compute summaries of monitoring data in window intervals within one minute. Further, the sample shows that framework was able to compute data originated from different cloud platforms.

RBC name	123 summation	123 count	123 time_bucket
backend-service	29	3	202,110,220,024
backend-service	118	3	202,110,220,025
backend-service	122	3	202,110,220,026
backend-service	109	3	202,110,220,027
backend-service	134	3	202,110,220,028
backend-service	116	3	202,110,220,029
backend-service	87	3	202,110,220,030
backend-service	90	3	202,110,220,031
backend-service	102	3	202,110,220,032
backend-service	113	3	202,110,220,033
backend-service	115	3	202,110,220,034

Figure 28: Computed Samples in SkyWalking

ABC service	ABC name	ABC platform	time	123 value
backend-service	bk_service.http_server_requests_seconds_sum.inc	Google	2021-11-08 10:42:58	-4.569702189
backend-service	bk_service.http_server_requests_seconds_count.inc	Google	2021-11-08 10:42:58	2,755.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Google	2021-11-08 10:42:52	4.605482581
backend-service	bk_service.http_server_requests_seconds_count.inc	Google	2021-11-08 10:42:52	-2,739.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Google	2021-11-08 10:42:52	0.0137598355
backend-service	bk_service.http_server_requests_seconds_count.inc	Google	2021-11-08 10:42:52	9.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Google	2021-11-08 10:42:42	0.010339955
backend-service	bk_service.http_server_requests_seconds_count.inc	Google	2021-11-08 10:42:42	7
backend-service	bk_service.http_server_requests_seconds_sum.inc	Digital_Ocean	2021-11-08 10:42:32	4.6096345765
backend-service	bk_service.http_server_requests_seconds_count.inc	Digital_Ocean	2021-11-08 10:42:32	-2,735.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Digital_Ocean	2021-11-08 10:42:31	4.6110085145
backend-service	bk_service.http_server_requests_seconds_count.inc	Digital_Ocean	2021-11-08 10:42:31	-2,734.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Digital_Ocean	2021-11-08 10:42:28	-4.5691330025
backend-service	bk_service.http_server_requests_seconds_count.inc	Digital_Ocean	2021-11-08 10:42:28	2,751.5
backend-service	bk_service.http_server_requests_seconds_sum.inc	Digital_Ocean	2021-11-08 10:42:17	4.6060327215

Figure 29: Computed Sample by Proposed Modules

CHAPTER 6

DISCUSSION

The efficacy evaluation shows that all designed modules of the framework successfully achieved their desired goals and meet the guiding DGM requirements. In this experiment, the framework modules were distributed between separate Kubernetes platforms in two different cloud environments. The distributed modules communicated with each other through a cluster of data distribution modules (DDM). The observed telemetry from the distributed microservices, such as performance metrics, interactions spans, and event logs, were collected and forwarded to the framework. The distributed data source modules (DSM) received telemetry data from both platforms and transformed it into a unified data model before publishing it to other components in the framework to consume. The other modules of the framework such as the data management module (DMM), the data stream analysis module (SAM) and the data inspection module (DIM) were able to successfully request streams of telemetry data to perform management and analysis tasks.

The designed mechanism of obtaining and transferring telemetry over TCP transport layer as non-blocking reactive data stream facilitates an innovative method to observe distributed microservices in multi-cloud environments. The experiment linked two different cloud platforms by connecting a cluster that consists of two RSocket-based DDM brokers. Microservice observed telemetry was able to be acquired by any framework module regardless of the cloud platform. For instance, a data management module (DMM) that resides in GCP was able to acquire observed telemetry from the two data source modules (DSM) in GCP and DOCP environments. Other existing frameworks and utilities have a limitation to observe microservices in one cloud platform. Famous and commonly used monitoring tools and frameworks, such as Apache SkyWalking and Prometheus, do not observe microservices across multiple clouds without replicating their installation in each platform.

In addition to observing and transferring performance metrics across cloud platforms, the experiment was able to retrieve and filter log messages and spans. The framework data management module (DMM) was able to store on-demand all or any of the telemetry types in

the designed PostgreSQL database. The data were summarized in real-time from streams of observed telemetry by the data streaming analysis module (SAM). Furthermore, the inspection module (DIM) was able to continuously detect anomalies on stored data or in real-time from in-flight data streams if an aggregated value violated a threshold. The proposed framework not only reports the violated metric but was also able to attach correlated telemetry from storage that was captured about the same anomaly time.

Furthermore, the second conducted experiment shows that the designed framework performed better compared to the existing Apache SkyWalking framework. Notice from the results that all proposed modules in the experiment used less memory than Apache SkyWalking. Furthermore, the CPU utilization by the proposed modules is more efficient in performing computations with more running and fewer waiting threads. More threads were waiting to perform tasks that made SkyWalking components to stay idle and not use the system resources efficiently. All proposed modules in the experiment kept a greater number of threads busy with a smaller number of blocked threads waiting to take turns. This allowed the proposed modules to continuously take advantage of available CPU time. Additionally, the proposed framework was able to evaluate and store more aggregated values. Apache SkyWalking buffers telemetry data and computes one summarized value every minute, while the proposed framework emits summarized values over moving time windows.

Both experiments illustrated the applicability of an extendable, effective, and innovative multi-cloud distributed observability framework. As shown in Figure 30 the framework is equipped with modules to efficiently observe, transfer, and perform complex computations tasks in observed telemetry data. The framework can acquire observed telemetry data from containerized microservices deployed in Kubernetes environments across different cloud platforms. Further, the designed framework can observe all three pillars of telemetry data such as performance metrics, event logs, and interactions spans.



Figure 30: Framework Modules Tasks

CHAPTER 7

CONCLUSIONS

The challenge of collecting and analyzing telemetry data from multiple cloud platforms needs a scalable and efficient observability system. This challenge inspires the work in this thesis to contribute an architecture design for a distributed and extensible observability framework. The designed framework is guided to address area of concerns that are outlined by the decision guidance model (DGM) for monitoring microservices. The framework uses contemporary techniques in reactive stream processing to propose innovative methods to collect, transfer, and analyze microservices metrics, logs, and trace spans data.

The framework is equipped with a data source module (DSM) to receive the observed telemetry, a data management module (DMM) to manage storing and retrieval of data, a data streaming analysis module (SAM) to process and analyze telemetry streams, and a data inspection module (DIM) to inspect telemetry for anomalies. Communication between the framework modules across the cloud is facilitated by a cluster of data distributing modules (DDM). The efficacy evaluation suggested successful accomplishment of the desired observability tasks by the design framework. Being able to observe services across Kubernetes cloud platforms contributes a unique observability framework method when compared to existing frameworks and tools. The efficiency evaluation of the framework has shown better performance in terms of system resources consumption such as CPU, memory, and threads utilization. The designed framework provides systems operation practitioners with an efficient mechanism to link together dispersed systems to observed hosted microservices.

Researchers can also use the proposed architecture to further design and implement solutions that realize other highlighted area of concerns in DGM. For example, more design options could be suggested to address monitored data presentation and reporting concerns and provide more analytic capabilities, such as machine learning and operation intelligence (AIOPs). Furthermore, additional consideration could be given to other types of telemetry data, such as frontend interactions of users (clicks pattern / number of visits / sessions information / etc.) and business-related key performance indicators (KPI). Researchers can also try to use different programming languages, difference development frameworks, or transferring data

over different transport protocols to implement the proposed observability modules and methods to further enhance and evaluate system performance and runtime resources consumption.

REFERENCES

- Al Jawarneh, I. M., Bellavista, P., Bosi, F., Foschini, L., Martuscelli, G., Montanari, R., & Palopoli, A. (2019). Container Orchestration Engines: A Thorough Functional and Performance Comparison. *IEEE International Conference on Communications, 2019-May*. <https://doi.org/10.1109/ICC.2019.8762053>
- Alhamazani, K., Ranjan, R., Mitra, K., Jayaraman, P. P., Huang, Z., Wang, L., & Rabhi, F. (2014). CLAMS: Cross-layer multi-cloud application monitoring-as-a-service framework. *Proceedings - 2014 IEEE International Conference on Services Computing, SCC 2014*, iii, 283–290. <https://doi.org/10.1109/SCC.2014.45>
- Amazon. (2022). *Amazon CloudWatch*. <https://aws.amazon.com/cloudwatch/>
- Cinque, M., Della Corte, R., & Pecchia, A. (2019a). Advancing monitoring in microservices systems. *Proceedings - 2019 IEEE 30th International Symposium on Software Reliability Engineering Workshops, ISSREW 2019*, 122–123. <https://doi.org/10.1109/ISSREW.2019.00060>
- Cinque, M., Della Corte, R., & Pecchia, A. (2019b). Microservices Monitoring with Event Logs and Black Box Execution Tracing. *IEEE Transactions on Services Computing*, 1374(c), 1–14. <https://doi.org/10.1109/TSC.2019.2940009>
- Datadog. (2022). *Monitoring and Security Platform for Cloud Applications*. <https://www.datadoghq.com/>
- Fazio, M., Celesti, A., Ranjan, R., Liu, C., Chen, L., & Villari, M. (2016). Open Issues in Scheduling Microservices in the Cloud. *IEEE Cloud Computing*, 3(5), 81–88. <https://doi.org/10.1109/MCC.2016.112>
- FuentBit. (2022). *An End to End Observability Pipeline*. <https://fluentbit.io/>
- Gibbs, S., Dokuka, O., & Olga Maciaszek. (2022a). *RSocket Broker Client*. <https://github.com/rsocket-broker/rsocket-broker-client>
- Gibbs, S., Dokuka, O., & Olga Maciaszek. (2022b). *RSocket Routing Broker*. <https://github.com/rsocket-broker/rsocket-broker>
- Google. (2022). *Google Cloud Monitoring*. <https://cloud.google.com/monitoring>

- Haselbock, S., & Weinreich, R. (2017). Decision guidance models for microservice monitoring. *Proceedings - 2017 IEEE International Conference on Software Architecture Workshops, ICSAW 2017: Side Track Proceedings*, 54–61. <https://doi.org/10.1109/ICSAW.2017.31>
- Haselböck, S., Weinreich, R., & Buchgeher, G. (2017). Decision guidance models for microservices - Service discovery and fault tolerance. *ACM International Conference Proceeding Series, Part F1305*. <https://doi.org/10.1145/3123779.3123804>
- Hevener, A., March, S., & Park, J. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <http://www.springerlink.com/index/pdf/10.1007/s11576-006-0028-8>
- Khan, A. (2017). Key Characteristics of a Container Orchestration Platform to Enable a Modern Application. *IEEE Cloud Computing*, 4(5), 42–48. <https://doi.org/10.1109/MCC.2017.4250933>
- Lewis, G. A., Lago, P., & Avgeriou, P. (2016). A decision model for cyber-foraging systems. *Proceedings - 2016 13th Working IEEE/IFIP Conference on Software Architecture, WICSA 2016*, 51–60. <https://doi.org/10.1109/WICSA.2016.38>
- Mansouri-Samani, M., & Sloman, M. (1993). Monitoring Distributed Systems: A Functional Model of Monitoring in Terms of the Generation, Processing, Dissemination, and Presentation of Information can Help Determine the Facilities Needed to Design and Construct Distributed Systems. *IEEE Network*, 7(6), 20–30. <https://doi.org/10.1109/65.244791>
- Mansouri, M.-S., & Sloman, M. (1987). Monitoring Distributed Systems. *ACM Transactions on Computer Systems (TOCS)*, 5(2), 121–150. <https://doi.org/10.1145/13677.22723>
- March, S. T., & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems*, 15(4), 251–266. [https://doi.org/10.1016/0167-9236\(94\)00041-2](https://doi.org/10.1016/0167-9236(94)00041-2)
- Microsoft. (2022). *Azure Monitor*. <https://azure.microsoft.com/en-us/services/monitor/>
- Noor, A., Jha, D. N., Mitra, K., Jayaraman, P. P., Souza, A., Ranjan, R., & Dustdar, S. (2019). A framework for monitoring microservice-oriented cloud applications in heterogeneous

- virtualization environments. *IEEE International Conference on Cloud Computing, CLOUD, 2019-July*, 156–163. <https://doi.org/10.1109/CLOUD.2019.00035>
- OpenTelemetry. (2022). *Tools, SDK and APIs Designed for Creation and Management of Telemetry Data such as Traces, Metrics, and Logs*. <https://opentelemetry.io/>
- Prometheus. (2022). *Open-source Systems Monitoring and Alerting Toolkit*. <https://prometheus.io/>
- Reactor, P. (2022). *Reactive Library based on Reactive Stream Specification for Building non-blocking applications on the JVM*.
- Rovnyagin, M. M., Odintsev, V. V., Fedin, D. Y., & Kuzmin, A. V. (2018). Cloud computing architecture for high-volume monitoring processing. *Proceedings of the 2018 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering, ElConRus 2018, 2018-Janua*, 361–365. <https://doi.org/10.1109/ElConRus.2018.8317107>
- RSocket. (2022). *Application protocol providing Reactive Streams semantics*. <https://rsocket.io/>
- SkyWalking. (2022). *Apache Foundation SkyWalking*. <https://skywalking.apache.org/>
- Vaishnavi, V., & Kuechler, B. (2004). Design Science Research in Information Systems Overview of Design Science Research. *Association for Information Systems*.

APPENDICES

APPENDIX A: USERS' MANUAL

Glossary

Term	Description
Monitoring	The process of collecting, aggregating, and analyzing metric data to improve awareness of your components' characteristics and behavior
Telemetry	The process of recording and transmitting the readings of an instrument.
Trace	The complete processing of a request. The trace represents the whole journey of a request as it moves through all the services or components of a distributed system.
Span	The primary building block of a distributed trace, representing an individual unit of work done in a distributed system.
Metric	A numeric representation of data measured over intervals of time.
Event Log	A discrete event that happened and logged over time. Can be in a plaintext, structured or binary format.
Kubernetes	A portable, extensible, open source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation.
Pod	The smallest deployable computing unit in Kubernetes. It represents a single instance of a running process in a cluster and encapsulates one or more containers.
Container	A standard unit of software that packages up code and all its dependencies, so that the application runs quickly and reliably from one computing environment to another.
Microservice	A loosely coupled and independently deployable smaller component, or service with its own bounded defined context own technology stack, and independent development life cycle.

How to Deploy

Scripts were created, as depicted in Figure 31, to assist with deploying the framework in a Kubernetes cloud platform. The platform artifacts are deployed as Kubernetes deployment objects and defaulted with one instance (Pod) for each component. As shown in the figure below, the installation scripts are organized in different folders, each named to reflect the artifact module. Users should have Kubernetes “kubectl” and the “Kustomize” CLI installed. Provided users have access to a Kubernetes cluster, to install any of the modules or the PostgreSQL database, users can navigate to the prospective folder and run the following command from a terminal.

```
$ kustomize build . | kubectl apply -f -
```

Users can start by creating first the database and then the distributor, followed by the DSM, DMM, DIM, and SAM modules. The distributor should expose a load balancer service for another distributor to join its cluster. Users must watch the distributor logs and make sure all modules are connected. User must replicate the same to deploy the DMM, DIM and SAM modules on another hosted Kubernetes platform.

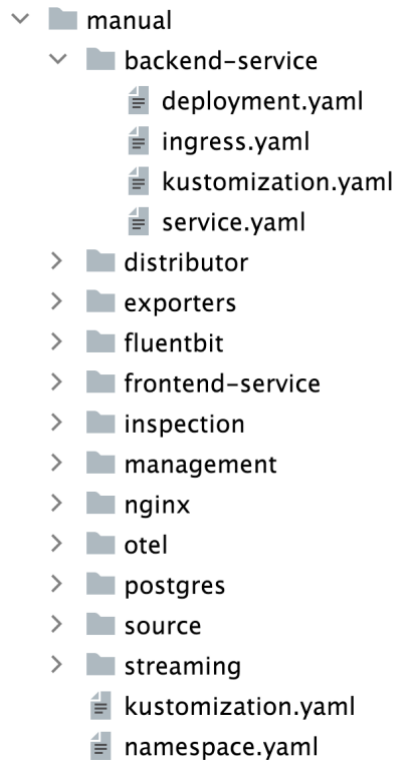


Figure 31: Deployment Artifact

As an example, the deployment of the framework artifacts in Digital Ocean and Google cloud Kubernetes platforms is described below.

Framework Experiment Setup

This experiment was conducted to evaluate the efficacy of the designed framework. First, all the five modules of the framework were packaged and built as docker container images. The containerized artifacts enable to deploy and run the framework in a Kubernetes platform, with the appropriate deployment manifest and adequate configuration. The framework could be deployed and distributed across multiple cloud environments; however, in this experiment only two cloud environments were used. The experiment used Kubernetes services offered by Google Cloud Platform (GCP) and Digital Ocean Cloud Platform (DOCP) Cloud. The decentralized feature of the framework enables different deployment topology of the modules. All modules of the framework in this setup, and as shown in Figure 32, were deployed in the DOCP environment, and only the DSM was deployed in the GCP environment. The framework modules were connected across the two cloud platforms by using a cluster of

DDM units. OTEL and Fluent-Bit agents were used in both platforms to obtain and feed the telemetry data to the framework through the DSM units.

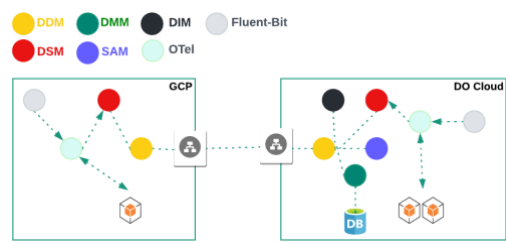


Figure 32: Experiment Setup

Error! Reference source not found.Figure 33Figure 34 show snapshots of the Kubernetes clusters used for the experiment in both GCP and DOCP cloud providers. GCP cluster consist of five nodes, each has 4 vCPU and 8 GB of RAM; and DOCP cluster has five nodes each with 4 vCPU and 16 GB of RAM.

Name	IP Address	Name ↑	Status
pool-research-c8wh7 8 GB / 25 GB Disk / AMS3 - Debian do-kub...	159.223.222.43	gke-cluster-research-default-pool-adfa28e8-18nh	✔ Ready
pool-research-c8whu 8 GB / 25 GB Disk / AMS3 - Debian do-kub...	159.223.222.254	gke-cluster-research-default-pool-adfa28e8-4tk5	✔ Ready
pool-research-c8wh8 8 GB / 25 GB Disk / AMS3 - Debian do-kub...	159.223.209.246	gke-cluster-research-default-pool-adfa28e8-7jc4	✔ Ready
pool-research-c8wh3 8 GB / 25 GB Disk / AMS3 - Debian do-kub...	159.223.222.30	gke-cluster-research-default-pool-adfa28e8-vmrq	✔ Ready
pool-research-c8whc 8 GB / 25 GB Disk / AMS3 - Debian do-kub...	159.223.222.109	gke-cluster-research-default-pool-adfa28e8-zj5p	✔ Ready

Figure 33: DOCP - 5 nodes x 4 vCPU x 8 GB RAM

Figure 34: GCP - 5 nodes x 4 vCPU x 16 GB RAM

Artifacts Deployed in Digital Ocean

The figures below depict the deployed framework modules in the Digital Ocean Kubernetes platform. The snapshots show a Postgres database, DDM, DIM, DMM, DSAM, and DSM. The snapshots also depict instances of deployed agents to collect generated telemetry data. The agents were deployed as a Kubernetes Daemon Set. That is, an instance of each agent is running on every Kubernetes cluster node.

Workloads / Deployments

Deployments

Name	Labels	Status	Age	Containers	Selector
postgres	app:postgres	1/1	8d	postgres	app:postgres
telemetry-distributor	app:telemetry-distributor	1/1	3d	telemetry-distributor	app:telemetry-distributor
telemetry-inspection	app:telemetry-inspection	1/1	3d	telemetry-inspection	app:telemetry-inspection
telemetry-management	app:telemetry-management	1/1	3d	observer-collector	app:telemetry-management
telemetry-source	app:telemetry-source	1/1	3d	telemetry-source	app:telemetry-source
telemetry-streaming	app:telemetry-inspection	1/1	3d	telemetry-streaming	app:telemetry-inspection

Daemon Sets

Name	Labels	Desired	Current	Ready	Up-To-Date	Age
fluent-bit	app:fluent-bit dep:n2 2+	4	4	4	4	12d

Daemon Sets

Name	Labels	Desired	Current	Ready	Up-To-Date	Age
otel-collector	app:otel component:otel-collector	4	4	4	4	9d

Artifacts Deployed in Google Cloud Platform

The figures below depict samples of deployed artifacts in a Google Cloud Platform's Kubernetes cluster. The snapshots show DDM, DSM, and instances of deployed agents to collect generated telemetry data for the platform. The agents were also deployed as a Kubernetes Daemon Set.

Workloads / Deployments

Deployments

Name	Labels	Status	Age	Containers	Selector
telemetry-distributor	app:telemetry-distributor	1/1	3d	telemetry-distributor	app:telemetry-distributor
telemetry-source	app:telemetry-source	1/1	3d	telemetry-source	app:telemetry-source

Daemon Sets

Name	Labels	Desired	Current	Ready	Up-To-Date	Age
fluent-bit	app:fluent-bit dep:n2 2+	4	4	4	4	8d

Daemon Sets

Name	Labels	Desired	Current	Ready	Up-To-Date	Age
otel-collector	app:otel component:otel-collector	4	4	4	4	8d

Additionally, three services were distributed across the two platforms to pose as a workload to observe by the framework. A Frontend service and a middle service were deployed in DO cloud, and a backend service was deployed in GCP. The Scenario is for the frontend service to call the backend service through the middle service. This scenario was subjected to a period of a load test with up to 2000 requests by 100 concurrent users.

The DSM module was able to successfully retrieve all telemetry data types from across the different cloud platforms. Below are sample of the captured data:

1- Logs sample:

```
data:{"telemetryType":"LOG","name":"log","service":"frontend-
service","labels":{"traceId":"d8605ca063ec9623b758d797688506d1","level":"INFO","fluent.tag":"kube.var.log.containers.frontend-
service-84747d8b-tcdnb_default_frontend-service-
7c2b8d75c75b56bc134ffefal38fd327fe28b2d3c69aba81e092b291d6464d23.log","thread":"or-http-epoll-
2","logClass":"io.agilehandy.front.FrontEndWeb","container_image":"docker.io/haybu/frontend-
service","namespace_name":"default","pod_name":"frontend-service-84747d8b-
tcdnb","spanId":"2af030543368ed35","container_name":"frontend-service","service":"frontend-
service","container_hash":"docker.io/haybu/frontend-service@sha256","host":"pool-research-
c8whu","docker_id":"7c2b8d75c75b56bc134ffefal38fd327fe28b2d3c69aba81e092b291d6464d23","pod_id":"f8c71ala-0b1c-48e7-a338-
8687989d6a96","on":"1"},"time":"2022-03-28T17:38:52.482-05:00","receivedTime":"2022-03-28T17:38:53.112796-
05:00","processedTime":"2022-03-28T17:38:53.112797-05:00","telemetryValue":{"value":"frontend service calls the remote middle
service","unit":"TEXT","stats":{"NaN"}}
```

2- Metrics sample:

```
data:{"telemetryType":"METRIC","name":"http_client_requests_seconds_sum","labels":{"app":"frontend-
service","k8s.pod.ip":"10.244.1.36","instance":"10.244.1.145:8080","kubernetes_pod_ip":"10.244.1.145","service.name":"kubernete
s-pods","scheme":"http","method":"GET","clientName":"middle-
service.default.svc.cluster.local","project":"research","pod_template_hash":"84747d8b","serviceName":"frontend-
service","uri":"/greet/Haytham","kubernetes_pod_name":"frontend-service-84747d8b-tcdnb","target":"default/frontend-
service","servicePlatform":"Digital-Ocean","kubernetes_namespace":"default","node":"pool-research-
c8whu","port":"8080","host.name":"10.244.1.145","job":"kubernetes-pods","outcome":"SUCCESS","status":"200"},"time":"2022-03-
28T17:56:22.575-05:00","receivedTime":"2022-03-28T17:56:24.522387-05:00","processedTime":"2022-03-28T17:56:24.522387-
05:00","telemetryValue":{"value":0.006722866,"unit":"S","stats":{"SUM"}}
```

3- Spans sample:

```
data:{"telemetryType":"SPAN","name":"HTTP GET","service":"frontend-
service","labels":{"mvc.controller.class":"FrontEndWeb","traceId":"2e323034456e910234f28f5424ef000e","k8s.pod.ip":"10.244.1.17
3","service.name":"frontend-
service","http.target":"/greet","kind":"SPAN_KIND_SERVER","http.path":"/greet","http.host":"134.209.133.84","parentId":"","tel
emetry.sdk.name":"opentelemetry","http.client_ip":"10.110.0.9","spanId":"d3c1cb4e35a84ed0","http.status_code":"","telemetry.sd
k.language":"java","mvc.controller.method":"remoteCall","http.user_agent":"ApacheBench/2.3","http.route":"/greet","telemetry.s
dk.version":"1.12.0","http.method":"GET","http.scheme":"http"},"time":"2022-03-28T17:40:11.251-05:00","receivedTime":"2022-03-
28T17:40:15.665589-05:00","processedTime":"2022-03-28T17:40:15.66559-
05:00","telemetryValue":{"value":2.43008574E8,"unit":"NANOSECONDS","stats":{"NaN"}}
```

The DMM was instructed to capture the raw telemetry data (metrics, logs, and trace spans) from the two different cloud platforms and save that to a database. As illustrated in Figure 35 **Error! Reference source not found.**, the DMM obtains the observed data from a cluster of DSM units that is distributed across the different cloud environments. Each telemetry

service	name	platform	instance	stats	time	value
frontend-service	jvm_gc_memory_promoted_bytes_total	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	57,419,360
frontend-service	jvm_thread_states_threads	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	0
frontend-service	application_ready_time_seconds	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	6.617
frontend-service	jvm_thread_states_threads	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	0
frontend-service	jvm_thread_states_threads	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	6
frontend-service	jvm_memory_allocated_bytes_total	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	15,003,191,920
frontend-service	jvm_classes_loaded_classes	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	10,820
frontend-service	jvm_classes_unloaded_classes_total	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	1,648,135,160,069
frontend-service	process_start_time_seconds	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	136
frontend-service	executor_queue_tasks	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	0
frontend-service	executor_queue_tasks	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	0
frontend-service	jvm_gc_max_data_size_bytes	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	429,260,800
frontend-service	process_uptime_seconds	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	102,422,633
frontend-service	jvm_memory_usage_after_gc_percent	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	0.1139750939
frontend-service	process_files_open_files	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	345
frontend-service	application_started_time_seconds	Digital-Ocean	10.244.0.188-8080	NaN	2022-03-25 14:46:22	6.61

Figure 42: Frontend service raw metrics

service	trace	span	level	time	value
frontend-service	0cc39f850d171b24158955f0c0be19	Bee979577de385a	INFO	2022-03-25 14:33:20	sleeping for 60 milliseconds
frontend-service	8a0676960c04707c4a1f0708306079a9	e93763c25a8a4d11	INFO	2022-03-25 14:33:20	sleeping for 70 milliseconds
frontend-service	b32383619b18148614a510b5989bad	de063aa33809c0	INFO	2022-03-25 14:33:20	frontend service calls the remote middle service
frontend-service	489432c3409815c345064ac08c681	32e2b5a77d932d	INFO	2022-03-25 14:33:20	sleeping for 60 milliseconds
frontend-service	b32383619b18148614a510b5989bad	de063aa33809c0	INFO	2022-03-25 14:33:20	sleeping for 71 milliseconds
frontend-service	489432c3409815c345064ac08c681	32e2b5a77d932d	INFO	2022-03-25 14:33:20	sleeping for 64 milliseconds
frontend-service	cac319272b4d44ac28b678a0abd1fb3	52b2748a0fca870	INFO	2022-03-25 14:33:20	sleeping for 64 milliseconds
frontend-service	cac319272b4d44ac28b678a0abd1fb3	52b2748a0fca870	INFO	2022-03-25 14:33:20	sleeping for 64 milliseconds
frontend-service	099c19a0b0c03065298a04d6e92108a	c5ae52683e0a178	INFO	2022-03-25 14:33:20	sleeping for 64 milliseconds
frontend-service	099c19a0b0c03065298a04d6e92108a	c5ae52683e0a178	INFO	2022-03-25 14:33:20	sleeping for 64 milliseconds
frontend-service	a279063f5759e1f32af842a07da9d	1011f0c100cda5a	INFO	2022-03-25 14:33:20	sleeping for 67 milliseconds
frontend-service	a279063f5759e1f32af842a07da9d	1011f0c100cda5a	INFO	2022-03-25 14:33:20	sleeping for 67 milliseconds
frontend-service	96919062c1344db382780c4b0c8bb38	01a6059a4099393	INFO	2022-03-25 14:33:20	sleeping for 74 milliseconds
frontend-service	96919062c1344db382780c4b0c8bb38	01a6059a4099393	INFO	2022-03-25 14:33:20	sleeping for 74 milliseconds
frontend-service	9e960c5ca8720bbac277eb1473ba3	1ccae7abed67ec01	INFO	2022-03-25 14:33:20	sleeping for 58 milliseconds
frontend-service	9e960c5ca8720bbac277eb1473ba3	1ccae7abed67ec01	INFO	2022-03-25 14:33:20	sleeping for 58 milliseconds

Figure 43: Frontend service logs

service	trace	span	class	request_path	time	value
frontend-service	5a0f7f04412280a5c02092c5a0b93a	5c3a533b897341	FrontEndWeb	/greet	2022-03-25 14:33:16	727,516,536
frontend-service	094b3c27b6343d9f979640c98755af	68b0706a50418b	FrontEndWeb	/greet	2022-03-25 14:33:16	451,310,027
frontend-service	094b3c27b6343d9f979640c98755af	746418b0c0ba37	FrontEndWeb	/greet	2022-03-25 14:33:16	389,583,467
frontend-service	5e7c29d4904d677ed113af1a272988b	e94d23a2d1ac1170	FrontEndWeb	/greet	2022-03-25 14:33:16	271,373,202
frontend-service	d0f1458962960a0130c5a4188ab1	d646c09f6150	FrontEndWeb	/greet	2022-03-25 14:33:16	328,545,570
frontend-service	d0f1458962960a0130c5a4188ab1	ee1637c445f0167	FrontEndWeb	/greet	2022-03-25 14:33:16	389,209,298
frontend-service	5e7c29d4904d677ed113af1a272988b	887a7c92727237	FrontEndWeb	/greet	2022-03-25 14:33:16	328,224,326
frontend-service	4aa45d6d4348f13075170b0d4820b6	37cc293a894e113	FrontEndWeb	/greet	2022-03-25 14:33:16	627,472,709
frontend-service	d38c5b29a6d710ca047c0e7a302f51	55aeca959a6090	FrontEndWeb	/greet	2022-03-25 14:33:16	497,985,580
frontend-service	003cc0d4eb7af6c0c07af9a008b239	d4e8ec203c0aee	FrontEndWeb	/greet	2022-03-25 14:33:16	125,787,281
frontend-service	003cc0d4eb7af6c0c07af9a008b239	8a0776932a329396	FrontEndWeb	/greet	2022-03-25 14:33:16	189,994,517
frontend-service	313aefcd2a5e79439f11e879480466	4d820ed3a214a80	FrontEndWeb	/greet	2022-03-25 14:33:16	976,482,089
frontend-service	ca466a23a29d9fbaccd5f10d2751a0	dc3e8030e2d286c	FrontEndWeb	/greet	2022-03-25 14:33:16	851,304,298
frontend-service	936c02b25a871a5e4b8a04473a081d	343958b0ab0226a	FrontEndWeb	/greet	2022-03-25 14:33:16	659,834,679
frontend-service	20c133a05d0c027701386460777f4b	ad6657a5d42050a	FrontEndWeb	/greet	2022-03-25 14:33:16	517,245,513
frontend-service	89a059b0a0c358376705e180a5b	dc04e6fac462a617	FrontEndWeb	/greet	2022-03-25 14:33:16	594,572,835
frontend-service	0909430e069696928c2375350846	e374aee029c23866	FrontEndWeb	/greet	2022-03-25 14:33:16	459,758,875

Figure 44: Frontend service invocation traces

As illustrated in Figure 45Figure 46Figure 47, the captured and stored metrics and spans data of all microservices from across different cloud environments could be presented (out of scope) to report related data in single graph. All logs could also be aggregated to present in one interface.

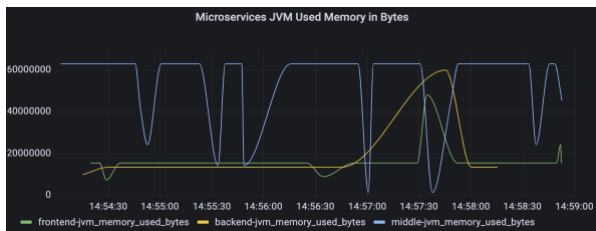


Figure 45: Microservices Captured Metrics

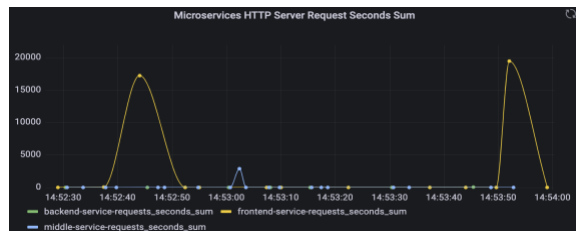


Figure 46: Microservices Captured Metrics

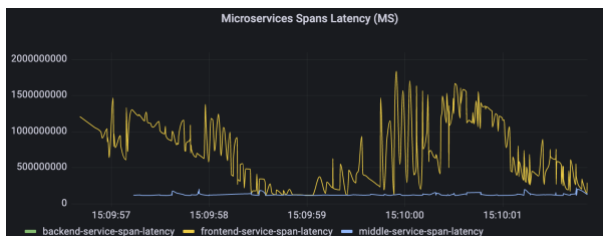


Figure 47: Microservices Captured Metrics

The stored data could also be summarized by the DMM aggregation functions. As illustrated in Figure 48, the DMM uses SAM units in the network to aggregate the data before storing it to a database.

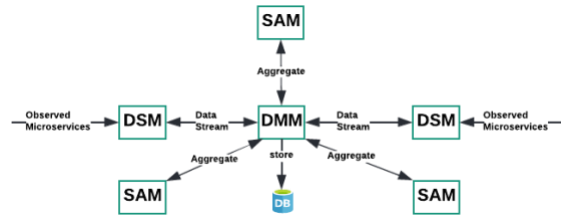


Figure 48: DMM Storage of Aggregated Data

For example, a user can send a request to DMM to obtain the average of the “jvm_memory_used_bytes” metric on the front-end-service every five seconds by using the URL [http://host:port/query/jvm_memory_used_bytes@frontend-service.avg\(5s\).since\(1d\)](http://host:port/query/jvm_memory_used_bytes@frontend-service.avg(5s).since(1d)). Sample of the result is listed below.

```
data:{ "telemetryType": "METRIC_SUMMARY", "name": "jvm_memory_used_bytes", "service": "frontend-service", "time": "2022-03-26T00:22:17.5Z", "receivedTime": "2022-03-26T00:25:10.627013Z", "processedTime": "2022-03-26T00:25:10.627022Z", "telemetryValue": { "value": 1.3517672E7, "unit": "B", "stats": "AVG" } }
```

The same could be retrieved from the SPAN data. For example, a user can continuously obtain the average latency from the frontend-service spans data by using a query with a URL that looks like: [http://host:port/query/spans@frontend-service{ "http.path": "/greet/Haytham" }.avg\(5s\).since\(1d\)](http://host:port/query/spans@frontend-service{). Note how a specific request path is specified with labels between two curly brackets.

```
data:{ "telemetryType": "SPAN_SUMMARY", "name": "span", "service": "backend-service", "time": "2022-03-26T03:26:20.5Z", "receivedTime": "2022-03-26T14:56:46.018708Z", "processedTime": "2022-03-26T14:56:46.018736Z", "telemetryValue": { "value": 1.4404938E7, "unit": "NaN", "stats": "AVG" } }
```

The DDM could be also used to store aggregated values of either metrics or spans telemetry values. A user should define the metrics or spans computation rules and set the “computed” flag to “true”. For instance, computation rules for the three simulating microservices (backend-service, middle-service, and frontend-service) were configured as listed below. The rules direct the DDM to compute the rate of increase in values of the “http_server_request_seconds_sum” and “http_server_request_seconds_count” metrics every one minute with a moving time window of five seconds.

```
telemetry:
  ingestion:
    computed: true
    rules:
      - name: backend-service.http_server_requests_seconds_sum.inc
        query: http_server_requests_seconds_sum@backend-service.increase(1m).slide(5s)
      - name: backend-service.http_server_requests_seconds_count.inc
        query: http_server_requests_seconds_count@backend-service.increase(1m).slide(5s)
      - name: middle-service.http_server_requests_seconds_sum.inc
        query: http_server_requests_seconds_sum@middle-service.increase(1m).slide(5s)
      - name: middle-service.http_server_requests_seconds_count.inc
        query: http_server_requests_seconds_count@middle-service.increase(1m).slide(5s)
      - name: frontend-service.http_server_requests_seconds_sum.inc
        query: http_server_requests_seconds_sum@frontend-service.increase(1m).slide(5s)
      - name: frontend-service.http_server_requests_seconds_count.inc
        query: http_server_requests_seconds_count@frontend-service.increase(1m).slide(5s)
```

To start ingesting the aggregated metrics into a database, a user can issue a command like the one below:

```
$ echo '{"category":"METRIC","batch":20}' | http host:port/ingest/signals/start
```

Because the aggregated values slide five seconds every one minute, this would result in ingestion rate of 12 record at a time (60 seconds divided by 5 seconds), as shown in the portion of the output log below (Figure 49). And Figure 50 lists a sample of the aggregated metrics as stored in the database.

```
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
12 number of telemetry records inserted!
```

Figure 49: Aggregated Metrics Ingestion Rate

service	name	labels	instance	unit	stats	time	value
backend-service	backend-service.http_server_requests_seconds_sum.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-196.0658472535
backend-service	backend-service.http_server_requests_seconds_count.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-5.042
middle-service	middle-service.http_server_requests_seconds_sum.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:07	17.2101519032
middle-service	middle-service.http_server_requests_seconds_count.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:07	129.3333333333
frontend-service	frontend-service.http_server_requests_seconds_sum.inc	("app":"frontend-service"	10.244.0.188-8080	S	INCREASE	2022-03-26 10:21:10	0.0023635074
frontend-service	frontend-service.http_server_requests_seconds_count.inc	("app":"frontend-service"	10.244.0.188-8080	S	INCREASE	2022-03-26 10:21:10	1.75
backend-service	backend-service.http_server_requests_seconds_sum.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-196.0658472535
backend-service	backend-service.http_server_requests_seconds_count.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-5.042
middle-service	middle-service.http_server_requests_seconds_sum.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:07	20.6609009862
middle-service	middle-service.http_server_requests_seconds_count.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:07	155.5
frontend-service	frontend-service.http_server_requests_seconds_sum.inc	("app":"frontend-service"	10.244.0.188-8080	S	INCREASE	2022-03-26 10:21:10	0.0023635074
frontend-service	frontend-service.http_server_requests_seconds_count.inc	("app":"frontend-service"	10.244.0.188-8080	S	INCREASE	2022-03-26 10:21:10	1.75
backend-service	backend-service.http_server_requests_seconds_sum.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-196.0658472535
backend-service	backend-service.http_server_requests_seconds_count.inc	("app":"backend-service"	10.68.3.9-8080	S	INCREASE	2022-03-26 10:21:15	-5.042
middle-service	middle-service.http_server_requests_seconds_sum.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:18	20.6575931693
middle-service	middle-service.http_server_requests_seconds_count.inc	("app":"middle-service"	10.244.1.50-8080	S	INCREASE	2022-03-26 10:21:18	154.5
frontend-service	frontend-service.http_server_requests_seconds_sum.inc	("app":"frontend-service"	10.244.0.188-8080	S	INCREASE	2022-03-26 10:21:37	6.1224785827494

Figure 50: Sample of the Stored Aggregated Metrics

A user can then query the aggregated metrics data through a URL that specifies a metric name and a service name like: http://localhost:port/query/backend-service.http_server_requests_seconds_sum.inc. This results in an output that looks like the sample below.

```
data:{"telemetryType":"METRIC","name":"frontend-service.http_server_requests_seconds_sum.inc","service":"frontend-service","labels":{"app":"frontend-service","job":"kubernetes-pods","uri":"/actuator/health/path","node":"pool-research-c8wh7","port":"8080","method":"GET","scheme":"http","status":"200","target":"default/frontend-service","outcome":"SUCCESS","project":"research","instance":"10.244.0.188:8080","exception":"None","host.name":"10.244.0.188","k8s.pod.ip":"10.244.1.36","serviceName":"frontend-service","service.name":"kubernetes-pods","servicePlatform":"DigitalOcean","kubernetes_pod_ip":"10.244.0.188","pod_template_hash":"6b9fd5b5b9","kubernetes_pod_name":"frontend-service-6b9fd5b5b9-qkb7h","kubernetes_namespace":"default"},"time":"2022-03-26T16:21:37.423Z","receivedTime":"2022-03-26T16:22:03.514414Z","processedTime":"2022-03-26T16:22:03.514416Z","telemetryValue":{"value":6603.262094158749,"unit":"S","stats":{"NaN"}}
```

Furthermore, the SAM enables a user to obtain live streams of hot (live) stream of raw and aggregated telemetry data. A user can query a SAM service with the same query string format that is used with the DMM (to query cold or stored data). For example, a request through <http://localhost:port/query/spans>, or <http://localhost:port/query/logs>, or <http://localhost:port/query/metrics> would result in a stream of raw telemetry data of spans, logs and metrics respectively. The output below is a sample of a stream of raw span data that was pulled using a typical URL.

```
data:{"telemetryType":"SPAN","name":"HTTP GET","service":"frontend-service","labels":{"traceId":"24530c82bacd3405f44aa0f7447e7f1","k8s.pod.ip":"10.244.0.207","service.name":"frontend-service","kind":"SPAN_KIND_CLIENT","http.path":"/greet/Haytham","http.uri":"http://middle-
```

```
service.default.svc.cluster.local:8080/greet/Haytham", "parentId": "d38f289737f2be53", "telemetry.sdk.name": "opentelemetry", "spanId": "07ab48c827462ef3", "http.status_code": "", "telemetry.sdk.language": "java", "http.route": "/greet/Haytham", "telemetry.sdk.version": "1.12.0", "http.method": "GET", "time": "2022-03-26T16:28:29.891Z", "receivedTime": "2022-03-26T16:28:34.934913Z", "processedTime": "2022-03-26T16:28:34.934916Z", "telemetryValue": {"value": 1.67451425E8, "unit": "NANOSECONDS", "stats": "NaN"}}
```

A user can also pull a continuous hot stream of summarized data using queries through URL that specify a target service, span or name of a metric, aggregation function with a time window, optionally a starting period in the past, and optionally to interpolate the data to fill missing instances with average values. For example, the result below is the output of pulling the span latency average every second of a microservice called “backend-service” through the URL [http://host:port/query/spans@backend-service.avg\(1s\)](http://host:port/query/spans@backend-service.avg(1s))

```
data: {"telemetryType": "SPAN_SUMMARY", "name": "on-backend", "service": "backend-service", "labels": {"traceId": "44a1385bc01ef4f542258bf2940adc60", "spanId": "7bb69ad41a2ae783", "k8s.pod.ip": "10.68.2.8", "telemetry.sdk.language": "java", "service.name": "backend-service", "method": "onBackend", "kind": "SPAN_KIND_INTERNAL", "telemetry.sdk.version": "1.12.0", "class": "MyController", "telemetry.sdk.name": "opentelemetry"}, "time": "0001-01-01T00:00:00Z", "receivedTime": "2022-03-26T17:08:13.120431Z", "processedTime": "2022-03-26T17:08:13.964684Z", "telemetryValue": {"value": 114491.0, "unit": "NANOSECONDS", "stats": "AVG"}}
```

And the output example below is an example of when continuously querying the average per second of the “jvm_memory_used_bytes” metric from the “backend-service” URL: [http://localhost:port/query/jvm_memory_used_bytes@backend-service.avg\(1s\)](http://localhost:port/query/jvm_memory_used_bytes@backend-service.avg(1s))

```
data: {"telemetryType": "METRIC_SUMMARY", "name": "jvm_memory_used_bytes", "service": "backend-service", "labels": {"app": "backend-service", "area": "nonheap", "k8s.pod.ip": "10.68.1.13", "instance": "10.68.3.9:8080", "kubernetes.pod.ip": "10.68.3.9", "service.name": "kubernetes-pods", "scheme": "http", "project": "research", "pod_template_hash": "546f86887c", "serviceName": "backend-service", "kubernetes.pod.name": "backend-service-546f86887c-2z9vt", "target": "default/backend-service", "servicePlatform": "Cluster-1", "kubernetes.namespace": "default", "node": "gke-cluster-research-default-pool-adfa28e8-vmrq", "port": "8080", "host.name": "10.68.3.9", "id": "CodeHeap 'non-profiled nmethods'", "job": "kubernetes-pods"}, "time": "0001-01-01T00:00:00Z", "receivedTime": "2022-03-26T17:16:17.632646Z", "processedTime": "2022-03-26T17:16:18.587749Z", "telemetryValue": {"value": 2.8527738E7, "unit": "B", "stats": "AVG"}}
```

Finally, the DIM facilitates detecting anomalies in metrics telemetry data. The sample output below was obtained by using the URL <http://localhost:port/metrics/alert> to request detecting anomalies in metrics based on threshold condition rules. The example below also shows an example of a threshold condition to detect a time moving average every minute of the “http_server_requests_seconds_sum” metrics in the backend-service. The threshold in the example should not exceed the value of 270 seconds once every five seconds. The rule instructs the detector to pause detecting for 3 seconds after each check. Note, the detected anomaly in the sample output below includes correlated logs and spans data that occur about the same time.

```
rules:
  ingestion_enabled: true
  source: stream
  conditions:
    - http_server_requests_seconds_sum@backend-service.twavg(1m) > 270 within 5s occurrence 1 sleep 3s

data: {"rule": "http_server_requests_seconds_sum@backend-service.twavg(1m) > 270 within 5s occurrence 1 sleep 3s", "violation": {"name": "http_server_requests_seconds_sum", "service": "backend-service", "instance": "192.168.1.65:9400", "host": "192.168.1.65", "port": 9400, "value": 2.7472047959683334E10, "unit": "s", "stats": "twavg", "time": "2022-03-26T14:28:38.581-05:00", "message": "Violation: 2.7472047959683334E10 GT 270.0", "observations": {"LOG": [{"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-8", "level": "INFO", "traceId": "79cda7464c35a51dfad3c51fe5b9296b", "spanId": "08ec14aa536245e", "time": "2022-03-26T14:28:33.319-05:00", "message": "backend service is last to be called in the greeting chain!"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-8", "level": "INFO", "traceId": "79cda7464c35a51dfad3c51fe5b9296b", "spanId": "08ec14aa536245e", "time": "2022-03-26T14:28:33.319-05:00", "message": "sleeping for 645 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-7", "level": "INFO", "traceId": "d80d2419d88f62a523491458032e1e3e", "spanId": "2241193b5ee80dd9", "time": "2022-03-26T14:28:33.152-05:00", "message": "sleeping for 331 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-11", "level": "INFO", "traceId": "a887f44c2363d3ae8d344efddb997377", "spanId": "d3cacdbb38955769", "time": "2022-03-26T14:28:23.782-05:00", "message": "sleeping for 260 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-
```

```

11", "level": "INFO", "traceId": "a887f44c2363d3ae8d344efddb997377", "spanId": "d3cacdbb38955769", "time": "2022-03-26T14:28:23.782-05:00", "message": "backend service is last to be called in the greeting chain!"), {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-7", "level": "INFO", "traceId": "59c2a3c2dbc048f85e3c14b7e85053ab", "spanId": "dd91276e129e0364", "time": "2022-03-26T14:28:23.385-05:00", "message": "sleeping for 266 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-7", "level": "INFO", "traceId": "59c2a3c2dbc048f85e3c14b7e85053ab", "spanId": "dd91276e129e0364", "time": "2022-03-26T14:28:23.385-05:00", "message": "backend service is last to be called in the greeting chain!"), {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-5", "level": "INFO", "traceId": "6bc0edc071affbf8a3141daelac03c9c", "spanId": "2f980356c6bac382", "time": "2022-03-26T14:28:23.357-05:00", "message": "backend service is last to be called in the greeting chain!"), {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-4", "level": "INFO", "traceId": "20772574718e7a5652e2056daf88c2d0", "spanId": "ba7f6cff7aceecc4", "time": "2022-03-26T14:28:23.357-05:00", "message": "sleeping for 176 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-5", "level": "INFO", "traceId": "6bc0edc071affbf8a3141daelac03c9c", "spanId": "2f980356c6bac382", "time": "2022-03-26T14:28:23.357-05:00", "message": "sleeping for 662 milliseconds"}, {"service": "backend-service", "className": "io.agilehandy.otel.MyController", "thread": "reactor-http-nio-4", "level": "INFO", "traceId": "20772574718e7a5652e2056daf88c2d0", "spanId": "ba7f6cff7aceecc4", "time": "2022-03-26T14:28:23.357-05:00", "message": "backend service is last to be called in the greeting chain!"), {"SPAN": [{"service": "backend-service", "instance": null, "traceId": "79cda7464c35a51dfad3c51fe5b9296b", "spanId": "08ec14aea536245e", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "7086f01faa03ce67", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:33.319-05:00", "latency": 6.47075311E8}, {"service": "backend-service", "instance": null, "traceId": "79cda7464c35a51dfad3c51fe5b9296b", "spanId": "08ec14aea536245e", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "7086f01faa03ce67", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:33.319-05:00", "latency": 6.47075311E8}, {"service": "backend-service", "instance": null, "traceId": "d80d2419d88f62a523491458032e1e3e", "spanId": "2241193b5ee80dd9", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "a6bddb847ef9a236", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:33.152-05:00", "latency": 3.36064519E8}, {"service": "backend-service", "instance": null, "traceId": "a887f44c2363d3ae8d344efddb997377", "spanId": "d3cacdbb38955769", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "b3aff48cff7blda2", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.782-05:00", "latency": 2.64645555E8}, {"service": "backend-service", "instance": null, "traceId": "a887f44c2363d3ae8d344efddb997377", "spanId": "d3cacdbb38955769", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "b3aff48cff7blda2", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.782-05:00", "latency": 2.64645555E8}, {"service": "backend-service", "instance": null, "traceId": "59c2a3c2dbc048f85e3c14b7e85053ab", "spanId": "dd91276e129e0364", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "f631fa7abel37c1b", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.385-05:00", "latency": 2.6838711E8}, {"service": "backend-service", "instance": null, "traceId": "59c2a3c2dbc048f85e3c14b7e85053ab", "spanId": "dd91276e129e0364", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "f631fa7abel37c1b", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.385-05:00", "latency": 2.6838711E8}, {"service": "backend-service", "instance": null, "traceId": "6bc0edc071affbf8a3141daelac03c9c", "spanId": "2f980356c6bac382", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "f38bacabee5b85a86", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.357-05:00", "latency": 6.66016503E8}, {"service": "backend-service", "instance": null, "traceId": "20772574718e7a5652e2056daf88c2d0", "spanId": "ba7f6cff7aceecc4", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "2a4c8af775425709", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.357-05:00", "latency": 1.81957701E8}, {"service": "backend-service", "instance": null, "traceId": "6bc0edc071affbf8a3141daelac03c9c", "spanId": "2f980356c6bac382", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "f38bacabee5b85a86", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.357-05:00", "latency": 6.66016503E8}, {"service": "backend-service", "instance": null, "traceId": "20772574718e7a5652e2056daf88c2d0", "spanId": "ba7f6cff7aceecc4", "className": "MyController", "method": null, "url": null, "httpStatusCode": null, "requestPath": "/greet/Haytham", "parentId": "2a4c8af775425709", "kind": "SPAN_KIND_SERVER", "peer": null, "time": "2022-03-26T14:28:23.357-05:00", "latency": 1.81957701E8}], "source": "STREAM"}

```

To detect anomalies, the data inspection module (DIM) sends the configured threshold rules to the connected SAMs to obtain either a raw or aggregated stream of data. The rules will be balanced between the connected SAMs. The SAM obtains the observed telemetry from all DSMs and uses the query string in the threshold rule to filter out and aggregate the data. As illustrated in Figure 51, the SAM then sends the results back to DIM which in turns continues to check the values against the configured thresholds. If a condition check is violated, the DIM will reach out to the DMM modules to try correlating any captured logs or spans data that obtained around the same time.

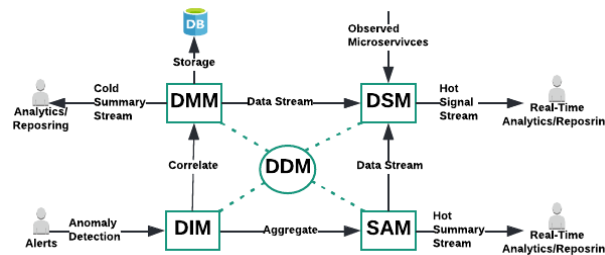


Figure 51: Framework Modules and Capabilities

How to Use the Framework Modules

The framework consists of five modules, the Data Distributor Module (DDMM), the Data Management Module (DMM), the Data Source Module (DSM), the Data Inspection Module (DIM), and the Data Streaming Analysis Module (SAM).

DSM: Data Source Module

The DSM is used to retrieving the raw telemetry signals by other modules in the framework. A user can also directly pull stream of raw telemetry data through the URL <http://localhost:8585/telemetry/metrics/spans/logs>. This URL would stream data with output that looks like the following:

Raw Spans:

```
data:{"telemetryType":"SPAN","name":"on-backend","service":"backend-
service","labels":{"traceId":"e521e77641775e46efb1e498954554d3","spanId":"1878c68f0876def5","telemetry.sdk.language":"java","service.name
":"backend-
service","method":"onBackend","kind":"SPAN_KIND_INTERNAL","telemetry.sdk.version":"1.12.0","class":"MyController","telemetry.sdk.name":"o
pentelemetry"},"time":"2022-03-21T20:26:45.492-05:00","receivedTime":"2022-03-21T20:26:47.348435-05:00","processedTime":"2022-03-
21T20:26:47.34844-05:00","telemetryValue":{"value":1044872.0,"unit":"NANOSECONDS","stats":"NaN"}}
data:{"telemetryType":"SPAN","name":"on-backend","service":"backend-
service","labels":{"traceId":"27658c940b7f1d9fd9f798a1bcc5fb8","spanId":"6e7ab32c575f69e3","telemetry.sdk.language":"java","service.name
":"backend-
service","method":"onBackend","kind":"SPAN_KIND_INTERNAL","telemetry.sdk.version":"1.12.0","class":"MyController","telemetry.sdk.name":"o
pentelemetry"},"time":"2022-03-21T20:27:00.497-05:00","receivedTime":"2022-03-21T20:27:02.334047-05:00","processedTime":"2022-03-
21T20:27:02.334053-05:00","telemetryValue":{"value":1889672.0,"unit":"NANOSECONDS","stats":"NaN"}}
```

Raw Logs

```
data:{"telemetryType":"LOG","name":"log","service":"backend-
service","labels":{"traceId":"e1bd9eal4f2eefcf76d61f4fa2dla484","spanId":"8833f2b50639c47a","level":"INFO","service":"backend-
service","fluent.tag":"local","thread":"scheduling-1","logClass":"io.agilehandy.otel.MyController","on":"34371"},"time":"2022-03-
21T15:28:40.555-05:00","receivedTime":"2022-03-21T20:28:47.25676-05:00","processedTime":"2022-03-21T20:28:47.256765-
05:00","telemetryValue":{"value":" Performing some logging on backend service with random number 6","unit":"TEXT","stats":"NaN"}}
```

Raw Metrics

```
data:{"telemetryType":"METRIC","name":"jvm_memory_committed_bytes","service":"backend-
service","labels":{"area":"nonheap","servicePlatform":"local","instance":"192.168.1.65:9400","service.name":"backend-service-collector-
scraper","scheme":"http","port":"9400","host.name":"192.168.1.65","id":"CodeHeap 'non-nmethods'","job":"backend-service-collector-
scraper","serviceName":"backend-service"},"time":"2022-03-21T20:41:57.074-05:00","receivedTime":"2022-03-21T20:41:59.330554-
05:00","processedTime":"2022-03-21T20:41:59.330554-05:00","telemetryValue":{"value":"2555904.0","unit":"B","stats":"NaN"}}
```

DMM: Data Management Module

A user can POST to a Restful endpoint <http://host:port/ingest/signals/start> to start ingesting raw telemetry into a data store (like PostgreSQL database). A user can post a request to specify type of telemetry, service name, starting time, and ingestion rate. The example below shows how to request ingesting all telemetry types (interaction spans, metrics, and event logs) with a rate of 500 records at a time. Figure 52Figure 53Figure 54 display snapshots of the stored metrics, logs, and spans respectively.


```
$ echo '{"category": "ALL", "batch": 500}' | http POST localhost:port/ingest/signals/start
```

src service	src name	labels	src instance	src unit	src stats	time	value
backend-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-nmethods", "job": "back 192.168.1.65:9400	BY	NaN		2022-03-22 14:13:41	18,874,368
middle-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-profiled nmethods", "job": "192.168.1.65:9300	BY	NaN		2022-03-22 14:13:41	7,286,784
middle-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-profiled nmethods", "job": "192.168.1.65:9300	BY	NaN		2022-03-22 14:13:41	14,680,064
middle-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-profiled nmethods", "job": "192.168.1.65:9300	BY	NaN		2022-03-22 14:13:41	312,475,648
middle-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-profiled nmethods", "job": "192.168.1.65:9300	BY	NaN		2022-03-22 14:13:41	2,555,904
middle-service	jvm_memory_committed_bytes	{ "id": "CodeHeap non-profiled nmethods", "job": "192.168.1.65:9300	BY	NaN		2022-03-22 14:13:41	318,767,104
frontend-service	jvm_buffer_count_buffers	{ "id": "mapped", "job": "frontend-service-scray 192.168.1.65:9200	NaN	NaN		2022-03-22 14:13:40	20
frontend-service	jvm_buffer_count_buffers	{ "id": "mapped", "job": "frontend-service-scray 192.168.1.65:9200	NaN	NaN		2022-03-22 14:13:40	0
frontend-service	system_cpu_count	{ "job": "frontend-service-scraper", "port": "92: 192.168.1.65:9200	NaN	NaN		2022-03-22 14:13:40	16
frontend-service	jvm_memory_committed_bytes	{ "id": "Compressed Class Space", "job": "fronti 192.168.1.65:9200	BY	NaN		2022-03-22 14:13:40	385,875,968
frontend-service	jvm_memory_committed_bytes	{ "id": "Compressed Class Space", "job": "fronti 192.168.1.65:9200	BY	NaN		2022-03-22 14:13:40	239,075,328
frontend-service	jvm_memory_committed_bytes	{ "id": "Compressed Class Space", "job": "fronti 192.168.1.65:9200	BY	NaN		2022-03-22 14:13:40	51,408,896
frontend-service	jvm_memory_committed_bytes	{ "id": "Compressed Class Space", "job": "fronti 192.168.1.65:9200	BY	NaN		2022-03-22 14:13:40	12,845,056

Figure 52: Example of Stored Raw Metrics data

src service	src trace	src span	labels	src class	src thread	src level	time	src value
backend-service	0d4c0628fb	125fc948a959	{ "on": "77853", "level": "EF io.agilehandy.otel.MyController	scheduling-1	ERROR		2022-03-22 09:13:34	some error occurs on backend service java.lang
backend-service	821f2d8ac6b	34b9993df7ce	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:13:59	Performing some logging on backend service v
backend-service	3e8b34f5363	58c599f1f7ea5	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:14:04	Performing some logging on backend service v
backend-service	0d9bb0264ac	05139a9e1581	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:14:39	Performing some logging on backend service v
backend-service	0e48e87118f	117138e22f20	{ "on": "77853", "level": "EF io.agilehandy.otel.MyController	scheduling-1	ERROR		2022-03-22 09:14:44	some error occurs on backend service java.lang
backend-service	76d0769f74d	97bd3b9ee7b9	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:15:09	Performing some logging on backend service v
backend-service	959ef62e8b6	871a72bd81ca	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:15:09	Performing some logging on backend service v
backend-service	b200ec60ed5	45881c1ddd3e	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:16:59	Performing some logging on backend service v
backend-service	16c521a3bea	f00c368fb8f16	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:17:14	Performing some logging on backend service v
backend-service	027521da3ae	7032442f6f114	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:17:24	Performing some logging on backend service v
backend-service	3d074986bbt	b95522c16449	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:15:14	Performing some logging on backend service v
backend-service	29b7450f3bd	292b9670a482	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:15:19	Performing some logging on backend service v
backend-service	b0d8b536fbd	39981a88fdd5	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:16:09	Performing some logging on backend service v
backend-service	b5e175a5753	3319b790be14	{ "on": "77853", "level": "IN io.agilehandy.otel.MyController	scheduling-1	INFO		2022-03-22 09:16:19	Performing some logging on backend service v
backend-service	c9532e71011	5284aded3b70	{ "on": "77853", "level": "EF io.agilehandy.otel.MyController	scheduling-1	ERROR		2022-03-22 09:15:24	some error occurs on backend service java.lang

Figure 53: Example of Stored Event Logs data

src service	src trace	src span	labels	src class	src method	time	value
backend-service	c00000799bbc10e0	20a5e744ed71662	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:15:34	163,533
backend-service	b5e175a575324d39	3319b790be1400b1	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:16:19	269,263
backend-service	c791042324ed140d	f5bcb8012725c86d	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:16:29	964,623
backend-service	43622df79c47636b	1d10eb7f63ae14a4	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:16:39	210,436
backend-service	b200ec60ed5770e5f	45881c1ddd3eba02	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:16:59	11,768,727
backend-service	5998b01d3867519e	e27202f957af705a	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:17:04	205,271
backend-service	3302ba3f0432b536f	06643aeaf137df7	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:18:59	219,469
backend-service	7d802d17ca847cbci	8db6842cdc88d7f3	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:19:39	272,733
backend-service	02752fda3ee9ef63d	7032442f6f1143c7	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:17:24	244,037
backend-service	b482f9f001fde63ff3	a24afb8a17a2cf	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:17:39	10,251,860
backend-service	812bd376db6bc322	d673a1ffc8c8f8df	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:19:09	1,551,585
backend-service	b4742a932bd4d0f3c	1ca133740ca9ab5f	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:19:49	1,373,437
backend-service	787cb76d41bee7f8c	4004f1c0aa053f56	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:19:14	19,474,724
backend-service	eb7204ec87e0c25d	066249f495861f4f	{ "kind": "SPAN_KIND_INTERNAL", "class": "MyController", "met MyController	onBackend		2022-03-22 14:19:54	171,297

Figure 54: Example of Stored Service Interaction Traces and Spans Data

The DMM module also allows to store summarized metrics based on configured rules. To enable ingesting aggregated metrics the “telemetry.ingestion.computed” property should be set to ‘true’. The aggregation rules could be defined in a file named ‘telemetry_rules.yml’ with the aggregation function, a service name, an aggregation window in seconds, and optionally an aggregation sliding window in seconds. The summary could be interpolated to account for missing scraping periods. The example below shows a rule to compute a summary of aggregated values every thirty seconds and sliding window of five seconds for the “http_server_request_seconds_sum” metrics from a service named “backend-service”. Missing values will be interpolated every five seconds (using average values by default). Figure 55 lists sample of stored summary metrics from this example rule. The DMM distributes the defined computation rules equally to all reachable data stream analysis modules (SAMs) through the

DDM cluster. All reachable SAM would then aggregate the time series telemetry data and return the results back to the DMM to persist into a data store.

```
telemetry:
  ingestion:
    rules:
      - name: service.http_server_requests_seconds_sum.summary
        query: http\_server\_requests\_seconds\_sum@backend-service.summary\(30s\).slide\(5s\).interpolate\(5s\)
```

service	name	labels	instance	unit	stats	time	value
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	SUM		2022-03-22 19:17:16	8,239,973,181,605.42
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	COUNT		2022-03-22 19:17:16	6
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE50		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE95		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	MIN		2022-03-22 19:17:16	30.420413786
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE90		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE99		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	AVG		2022-03-22 19:17:16	1,373,328,863,600.9033
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE75		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	MAX		2022-03-22 19:17:16	1,647,994,636,315
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	SUM		2022-03-22 19:17:21	8,239,973,206,620.425
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	COUNT		2022-03-22 19:17:21	6
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE50		2022-03-22 19:17:21	1,647,994,641,318
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE95		2022-03-22 19:17:21	1,647,994,641,318
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	MIN		2022-03-22 19:17:21	30.42478461
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE90		2022-03-22 19:17:21	1,647,994,641,318
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE99		2022-03-22 19:17:21	1,647,994,641,318
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	AVG		2022-03-22 19:17:21	1,373,328,867,770.0708
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	LE75		2022-03-22 19:17:21	1,647,994,641,318
backend-service	service.http_server_	{job:"backend-service-scraper",uri:"/a 192.168.1.65-9400	S	MAX		2022-03-22 19:17:21	1,647,994,641,318

Figure 55: Example of Stored Summarized Metric Data

On other hand, to retrieve the stored raw or summarized data, a user can send a GET request to a Restful endpoint in the form of [http://host:port/query/{query-string}](#). A query string can specify any of the following:

- (metric name)@(service name)({optional labels}).(aggregate function).(starting time)to query metrics. The example below retrieves the mean every minute of the JVM used buffer memory metric in bytes from a service named “backend-service” and starts the retrieval from generated data, starting two days ago. The metrics will be pulled from a service instance defined by an IP address.

[jvm buffer memory used bytes@backend-service{instance=192.168.1.240:9091}.avg\(1m\).since\(2d\)](#)

- Or, logs@(service name)({optional labels}).(aggregate function) to query logs. The example below retrieves logs

[logs@backend-service{instance=192.168.1.240:9091}.since\(2d\)](#)

- Or, spans@(service name)({optional labels}).(aggregate function), to query spans example:

[spans@backend-service{instance=192.168.1.240:9091}.since\(2d\)](#)

SAM: Data Stream Analysis Module

The data stream analysis module (SAM) could be used to aggregate **hot** streams of telemetry data. A user can send a query string to SAM by using a Restful endpoint in the form

of <http://host:port/query/{query}>. The format of the query string is same as the one used with the DMM. The example below illustrates a query string to retrieve a hot stream of the increase rate every 30 seconds of the “jvm_memory_used_type” metric from a service named “backend-service” over a slide time window of five seconds. Data is interpolated every second to account for missing values (using average values). This request results to retrieve data as shown in the sample below.

[jvm_memory_used_bytes@backend-service.increase\(30s\).slide\(5s\).interpolate\(1s\)](#)

```
data:{"telemetryType":"METRIC","name":"jvm_memory_used_bytes","service":"backend-service","labels":{"area":"nonheap","servicePlatform":"local","instance":"192.168.1.65:9400","service.name":"backend-service-scraper","scheme":"http","port":"9400","host.name":"192.168.1.65","id":"CodeHeap 'non-nmethods'","job":"backend-service-scraper","serviceName":"backend-service"},"time":"2022-03-22T19:54:55.433-05:00","receivedTime":"2022-03-22T19:55:00.267468-05:00","processedTime":"2022-03-22T19:55:28.840908-05:00","telemetryValue":{"value":-1.162033657142857E7,"unit":"B","stats":{"INCREASE"}}
```

The table below lists all aggregation functions that could be used to summarize the telemetry data.

Telemetry Function	Description
increase	Increase in time series data over time
rate	Rate of change in time series data over time
irate	Instance rate of change in time series data over time
difference	Difference in time series data over time
avg	Average of time series data over time
twavg	Time weighted average of time series data over time
interpolate	To interpolate missing time series data over time
count	Count number of observed telemetry items over time
min	Minimum values of observed telemetry over time
max	Maximum values of observed telemetry over time
sum	Total sum of observed telemetry over time
le50	50th percentile of observed telemetry values over time
le75	75th percentile of observed telemetry values over time
le90	90th percentile of observed telemetry values over time
le95	95th percentile of observed telemetry values over time
le99	99th percentile of observed telemetry values over time
summary	To get count, min, max, sum, avg, le50, le75, le90, le95, le99 of telemetry values over time

A user can also use the SAM to obtain all aggregated values at once. For example, a request to the URL [http://localhost:port/jvm_memory_used_bytes@backend-service.summary\(5s\)](http://localhost:port/jvm_memory_used_bytes@backend-service.summary(5s)) should get a summary of the 5-second interval aggregate values of the “jvm_memory_used_byte” metrics in a service called “backend-service”. The output below shows the sample of the result:

```
data:{"telemetryType":"METRIC_SUMMARY","name":"jvm_memory_used_bytes","service":"backend-service","labels":{"area":"nonheap","servicePlatform":"local","instance":"192.168.1.65:9400","service.name":"backend-service-scraper","scheme":"http","port":"9400","host.name":"192.168.1.65","id":"CodeHeap 'non-nmethods'","job":"backend-service-scraper","serviceName":"backend-service"},"time":"2022-03-26T15:48:46.015-05:00","receivedTime":"2022-03-26T15:48:49.465048-05:00","processedTime":"2022-03-26T15:48:53.377945-05:00","telemetryValue":{"value":{"LE99":2.196957142857143E7,"LE90":2.196957142857143E7,"MAX":2.196957142857143E7,"LE95":2.19695714285714
```



```
3E7,"LE50":2.196957142857143E7,"AVG":2.196957142857143E7,"COUNT":4.0,"SUM":8.787828571428572E7,"MIN":2.196957142857143E7,"LE75":2.196957142857143E7},"unit":"BY","stats":{"SUMMARY"}})
```

DIM: Data Inspection Module

The data inspection module (DIM) can help to detect telemetry metrics anomalies. If telemetry metric values violate certain condition and threshold, the DIM would report back the violation. The DIM would also try to correlate the violating metric with associated event logs and service interaction spans that took place at the time of the violation. A user can define rules like the ones shown in the example below and fire a Restful request (<http://localhost:port/metrics/alert>) to start detecting the anomalies. The example shows one of the conditional rules to validate that the time-weighted average every one minute does not exceed 0.002 seconds within a period of five seconds. The violation needs to occur two times within the five seconds. The rule pauses from checking for a period of three seconds to give the observed service a chance to correct itself. Note that, the detection could be applied to hot stream of observed telemetry, or to a cold stream from a stored telemetry raw data (controlled by the “source” property with either “storage” or “stream” values). However, to obtain the correlated logs and the span data to the anomaly, the framework should be simultaneously storing the telemetry data. The “ingestion enabled” flag indicates that metrics is being ingested by the DMM module at the time of anomalies. The example below shows how to configure anomaly detection rules, and the output shows an example of a detected anomaly from the stored metrics.

```
rules:
  ingestion_enabled: false
  source: storage
  conditions:
    - http_server_requests_seconds_sum@backend-service.since(1d).twavg(1m) > 0.002 within 5s occurrence 2 sleep 3s
    - http_server_requests_seconds_sum@backend-service.since(1d).le95(1m) > 0.03 within 10s

data:{
  "rule":"http_server_requests_seconds_sum@backend-service.since(1d).le95(1m) > 0.03 within 10s",
  "violation":{
    "name":"http_server_requests_seconds_sum",
    "service":"backend-service",
    "instance":null,
    "host":null,
    "port":0,
    "value":70.31610890054974,
    "unit":"s",
    "stats":{"le95","time":"2022-03-22T21:07:30-05:00"},"message":"Violation: 70.31610890054974 GT 0.03","observations":{"LOG":[{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"2233b2b0c4f7f498e88bfff0377336b34","spanId":"dlf7afe56c53fc2e","time":"2022-03-22T21:07:30.926-05:00","message":"Performing some logging on backend service with random number 0"},{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"86ale3c35e8cf5bbe811d83bca0be708","spanId":"61cf0719c162b30e","time":"2022-03-22T21:07:20.904-05:00","message":"Performing some logging on backend service with random number 1"},{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"84f9f79d64b6bc23035b74effb6e44bd","spanId":"8a2f54195a449776","time":"2022-03-22T21:07:15.899-05:00","message":"Performing some logging on backend service with random number 4"},{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"fc92efcd074ab42deff18ce5b9191d9","spanId":"33909b3309223a74","time":"2022-03-22T21:06:50.881-05:00","message":"Performing some logging on backend service with random number 1"},{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"c25fc5e67233a6e7ec78b30fdacef2","spanId":"4d758c39f043945b","time":"2022-03-22T21:06:35.872-05:00","message":"Performing some logging on backend service with random number 2"},{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"1df0e7c4d06a744fca12eba6a0a71e69","spanId":"ea97d41fc93a6f30","time":"2022-03-22T21:06:30.856-05:00","message":"Performing some logging on backend service with random number 0"}]},
    "SPAN":[{"service":"backend-service","instance":null,"traceId":"2233b2b0c4f7f498e88bfff0377336b34","spanId":"dlf7afe56c53fc2e","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:07:30.926-05:00","latency":152369.0},{"service":"backend-service","instance":null,"traceId":"86ale3c35e8cf5bbe811d83bca0be708","spanId":"61cf0719c162b30e","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:07:20.904-05:00","latency":139376.0},{"service":"backend-service","instance":null,"traceId":"84f9f79d64b6bc23035b74effb6e44bd","spanId":"8a2f54195a449776","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:07:15.899-05:00","latency":162808.0},{"service":"backend-service","instance":null,"traceId":"fc92efcd074ab42deff18ce5b9191d9","spanId":"33909b3309223a74","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:06:50.881-05:00","latency":152369.0},{"service":"backend-service","instance":null,"traceId":"c25fc5e67233a6e7ec78b30fdacef2","spanId":"4d758c39f043945b","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:06:35.872-05:00","latency":152369.0}, {"service":"backend-service","instance":null,"traceId":"1df0e7c4d06a744fca12eba6a0a71e69","spanId":"ea97d41fc93a6f30","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-22T21:06:30.856-05:00","latency":152369.0}]}
  }
```

```
Backend", "url": null, "httpStatusCode": null, "requestPath": null, "parentId": null, "kind": "SPAN_KIND_INTERNAL", "peer": null, "time": "2022-03-22T21:06:50.881-05:00", "latency": 225070.0}, {"service": "backend-service", "instance": null, "traceId": "c25fc5e67233a6e7ec78b30fdacef2", "spanId": "4d758c39f043945b", "className": "MyController", "method": "onBackend", "url": null, "httpStatusCode": null, "requestPath": null, "parentId": null, "kind": "SPAN_KIND_INTERNAL", "peer": null, "time": "2022-03-22T21:06:35.872-05:00", "latency": 191702.0}, {"service": "backend-service", "instance": null, "traceId": "1df0e7c4d06a744fca12eba6a0a71e69", "spanId": "ea97d41fc93a6f30", "className": "MyController", "method": "onBackend", "url": null, "httpStatusCode": null, "requestPath": null, "parentId": null, "kind": "SPAN_KIND_INTERNAL", "peer": null, "time": "2022-03-22T21:06:30.856-05:00", "latency": 11528564E7}], "source": "STORAGE"}
```

And the sample output below shows a detected anomaly from a hot (life) stream, however, the correlated telemetry (logs and spans) was obtained from the database storage. As shown, the captured metric anomaly includes the correlated spans and logs data that occur at the same time of the violation.

```
data:{"rule":"http_server_requests_seconds_sum@backend-service.twavg(1m) > 270 within 5s occurrence 1 sleep 3s","violation":{"name":"http_server_requests_seconds sum","service":"backend-service","instance":null,"traceId":"192.168.1.65:9400","host":"192.168.1.65","port":9400,"value":2.747204605637778E10,"unit":"s","stats":"twavg","time":"2022-03-26T14:27:01.716-05:00"},"message":"Violation: 2.747204605637778E10 GT 270.0","observations":{"LOG":[{"service":"backend-service","className":"io.agilehandy.otel.MyController","thread":"scheduling-1","level":"INFO","traceId":"023480760ac3c11b9c2fe20ae67a6d8d","spanId":"a964f5c0ee3bd72a","time":"2022-03-26T14:26:50.708-05:00","message":"Performing some logging on backend service with random number 1"}],"SPAN":[{"service":"backend-service","instance":null,"traceId":"023480760ac3c11b9c2fe20ae67a6d8d","spanId":"a964f5c0ee3bd72a","className":"MyController","method":"onBackend","url":null,"httpStatusCode":null,"requestPath":null,"parentId":null,"kind":"SPAN_KIND_INTERNAL","peer":null,"time":"2022-03-26T14:26:50.708-05:00","latency":641808.0}]},"source":"STREAM"}
```

APPENDIX B: SYSTEM TECHNICAL DOCUMENTATION

Framework Modules

The implemented modules of the framework are illustrated in the figure and listed in the table below.

- >  **common-module**
- >  **data-distributor-module**
- >  **data-inspection-module**
- >  **data-management-module**
- >  **data-source-module**
- >  **data-streaming-analysis-module**
- >  **data-telemetry-rsocket-client**

Module	Description
Common Module	Common data model and auxiliary components
Data Distributor Module (DDM)	A module to implement a routing broker to route telemetry data requests and stream across the cloud.
Data Inspection Module (DIM)	A module to run a continuous process to inspect anomalies based on configured rules.
Data Management Module (DMM)	A module to manage storing and retrieving telemetry data.
Data Source Module (DSM)	A module to act as a source to publish telemetry data all subscribed analytics modules.
Data Streaming Analysis Module (SAM)	A module to perform real-time data analysis in telemetry streams.
Data Telemetry RSocket Client	A client to facilitate all modules to interact and retrieve data from DSM.

Database Schema

The table below lists the descriptions of the three database tables and fields that were created in the framework to store event logs, metrics, and traces telemetry data.

Table	Description	Fields	
telemetry_metrics	Telemetry performance metrics table	service name labels instance unit stats time value	service name Telemetry name Key/value labels Service instance Unit of measure aggregation type Telemetry time Telemetry value (Double)
Telemetry_logs	Telemetry event logs table	service trace span labels class thread level time value	Service name Interaction trace Id Interaction span Id Key/value labels Invoked class name Running thread name Log level (INFO,WARN, etc.) Telemetry time Telemetry value (TEXT)
telemetry_spans	Telemetry service interaction spans table	service trace span labels parent instance class method url http_status_code request_path kind peer time value	Service name Interaction trace Id Interaction span Id Key/value labels Interaction parent Id Service instance Service class name Service method name Target URL HTTP status code Target service request path Interaction kind (service, client, etc.) Interacting party Telemetry time Telemetry value (Double)